

International Cybersecurity and Digital Forensics Academy (ICDFA)

Basic Computer Skills for Digital Forensics

SMTP Email Traffic Forensics

Student Name: Kafayat Omolara Animashawun

Student ID: 2025/FWSD/11317

Instructor Name: Aminu Idris

Date of Submission: 15/09/2026

EXECUTIVE SUMMARY

This presents a forensic examination of an SMTP email traffic capture using the supplied smtp.pcap packet capture. The analysis was performed in an isolated laboratory environment using a preserved original evidence file and a separate working copy to maintain evidence integrity.

The examination focused on identifying the SMTP communication between the email client and server, establishing the sequence of SMTP commands and responses, examining the authentication process, analysing Base64-encoded authentication values, reconstructing the email message from the TCP stream, and identifying relevant client, server and network metadata.

The analysis identified an SMTP session between the client 10.10.1.4 using ephemeral port 1470 and the SMTP server 74.53.140.153 on port 25. The session contained the expected SMTP transaction sequence, including the server greeting, EHLO, AUTH LOGIN, MAIL FROM, RCPT TO, DATA, and QUIT. The authentication exchange contained Base64-encoded values, demonstrating that Base64 was being used as an encoding mechanism rather than encryption.

The TCP stream was subsequently reconstructed to examine the email transaction and message contents. The reconstructed message provided email header information, including the sender and recipient, subject, message date, Message-ID, MIME information and the Microsoft Outlook client indicator. The capture also showed that the

SMTP server advertised STARTTLS; however, no TLS packets or STARTTLS command were identified in the captured session. This indicates that the analysed SMTP exchange remained visible in the packet capture rather than being protected by TLS during the observed transaction.

Overall, the analysis demonstrates how SMTP traffic can be examined at packet and TCP-stream level to establish communication details, authentication activity, email content and security characteristics while maintaining basic forensic evidence-handling procedures.

PURPOSE OF THE ASSIGNMENT

The purpose of this assignment is to perform a practical forensic examination of SMTP email traffic contained within a packet capture. The exercise is intended to demonstrate how network forensic tools can be used to identify and analyse an email communication session, interpret SMTP commands and responses, examine authentication activity, reconstruct email content and identify relevant network metadata.

The assignment also provides practical experience in handling digital evidence by preserving the supplied packet capture, creating a working copy for analysis, verifying file integrity using SHA-256 hashing, and documenting the findings obtained during the investigation.

OBJECTIVES OF THE ASSIGNMENT

The objectives of this assignment are to:

- Preserve the supplied SMTP packet capture and maintain the integrity of the original evidence.
- Create and verify a working copy of the packet capture for forensic analysis.
- Identify SMTP traffic and the TCP conversation associated with the email session.
- Establish the client and SMTP server involved in the communication.
- Identify the SMTP server port and client ephemeral port used during the session.
- Analyse the sequence of SMTP commands and server responses.
- Examine the SMTP authentication process and identify the use of AUTH LOGIN.

- Analyse the Base64-encoded authentication values while protecting sensitive credential information in the report.
- Reconstruct the SMTP TCP stream to examine the transmitted email.
- Identify relevant email headers, message information and email client indicators.
- Extract network metadata including IP addresses, TCP ports, MAC addresses and TCP stream information.
- Determine whether TLS or STARTTLS was actually used during the observed SMTP session.
- Document the forensic findings, limitations and security observations from the captured traffic.

METHODOLOGY OF THE ASSIGNMENT

The examination followed a structured network-forensic process. First, the supplied smtp.pcap file was acquired into the laboratory evidence directory and preserved as the original evidence. A separate working copy was then created so that analysis could be performed without modifying the original capture.

The SHA-256 hash of both the original evidence file and the working copy was calculated and compared. The matching hash values confirmed that the working copy was identical to the original evidence before analysis. Capture characteristics were also recorded using capinfos, including the number of packets, capture duration, timestamps, file size and other relevant metadata.

The working copy was opened in Wireshark for an initial examination of the captured traffic. An SMTP display filter was then applied to isolate the SMTP packets from the other network traffic. The TCP conversation associated with the SMTP session was also examined using TShark to establish the client, server, ports, frame counts and duration of the communication.

The SMTP packets were subsequently extracted using TShark fields to identify the SMTP commands, parameters and server response codes. This allowed the sequence of the email transaction to be established, from the initial 220 server greeting through authentication and message transmission to the final QUIT and 221 response.

The authentication exchange was examined separately to identify the Base64-encoded values transmitted during AUTH LOGIN. Base64 decoding was performed offline where required for forensic verification, while sensitive credential values were not reproduced in the final report.

The SMTP TCP stream was then reconstructed to examine the email message and its headers. Relevant fields such as the date, sender, recipient, subject, Message-ID, MIME information and email client indicator were identified. The network-layer and link-layer metadata associated with the communication was also examined, including IP addresses, TCP ports, TCP stream information and MAC addresses.

Finally, the capture was examined for evidence of TLS or STARTTLS. Although the SMTP server advertised STARTTLS as an available capability, the observed session was checked for an actual STARTTLS command and TLS packets. The results were documented together with the limitations of determining encryption solely from the advertised server capabilities.

TOOLS AND RESOURCES USED

The following tools and resources were used during the forensic examination:

Tool / Resource	Purpose
Kali Linux	Primary laboratory environment used to perform the packet capture analysis and forensic examination.
Wireshark 4.6.6	Used for graphical inspection and filtering of the packet capture, SMTP traffic analysis, TCP stream identification and examination of packet-level metadata.
TShark 4.6.6	Used for command-line extraction of TCP conversation statistics, SMTP commands and responses, authentication fields and network metadata.
Python 3.13.9	Used for offline Base64 decoding during the SMTP authentication analysis.

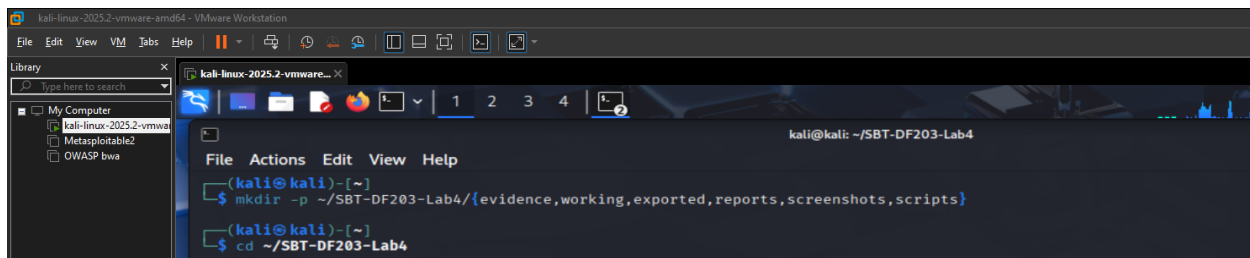
Tool / Resource	Purpose
sha256sum	Used to calculate SHA-256 hash values for verification of the original evidence and working copy.
capinfos	Used to obtain and record characteristics and metadata of the packet capture, including packet count, file size, capture duration and timestamps.
smtp.pcap	Supplied packet capture used as the primary digital evidence for the investigation.
SMTP Protocol	Analysed to identify the email server greeting, commands, responses, authentication process and message transmission sequence.
TCP Stream Reconstruction	Used to reassemble and examine the SMTP communication and reconstruct the transmitted email message and its headers.

Evidence Handling and Laboratory Preparation

The analysis workspace was created under the directory /home/kali/SBT-DF203-Lab4. The directory was structured into separate folders for original evidence, working copies, exported analysis artefacts, reports, screenshots and scripts. This structure was established before analysis commenced to support orderly evidence management and ensure that all laboratory outputs remain associated with the same case directory.

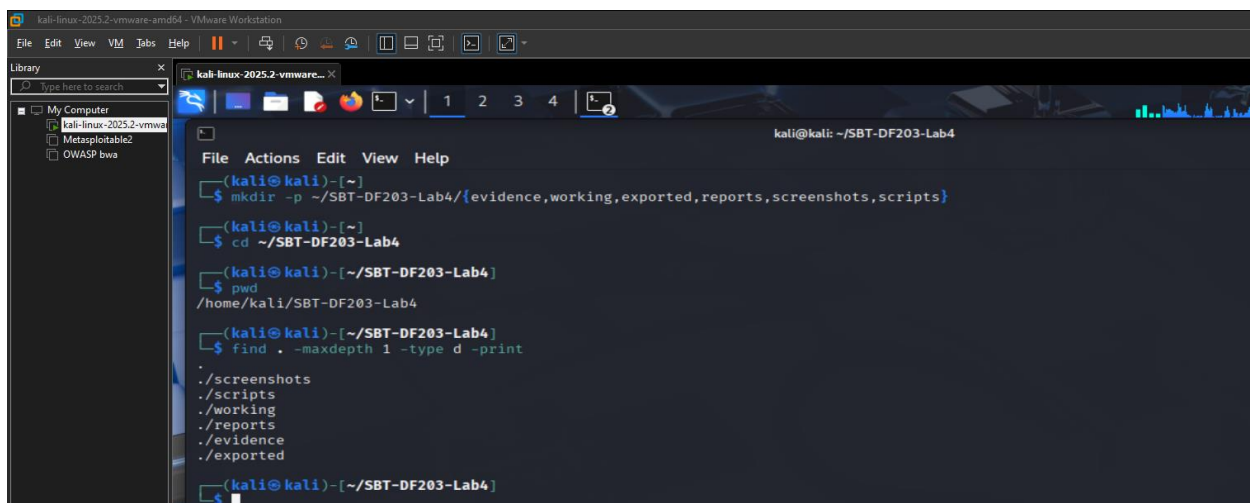
The evidence directory is reserved for the original supplied packet capture, while the working directory is intended for the verified copy used during analysis. The remaining directories will contain generated reports, screenshots, exported packet-analysis results and supporting scripts.

This approach preserves the original evidence while allowing analysis to be performed on a separate working copy, consistent with the laboratory's evidence-preservation requirements.



Initial Workspace Verification

The laboratory directory and its associated subdirectories were created successfully before the packet capture was downloaded. The working directory was then verified using the `pwd` and `find` commands to establish the analysis location and confirm the required folder structure.



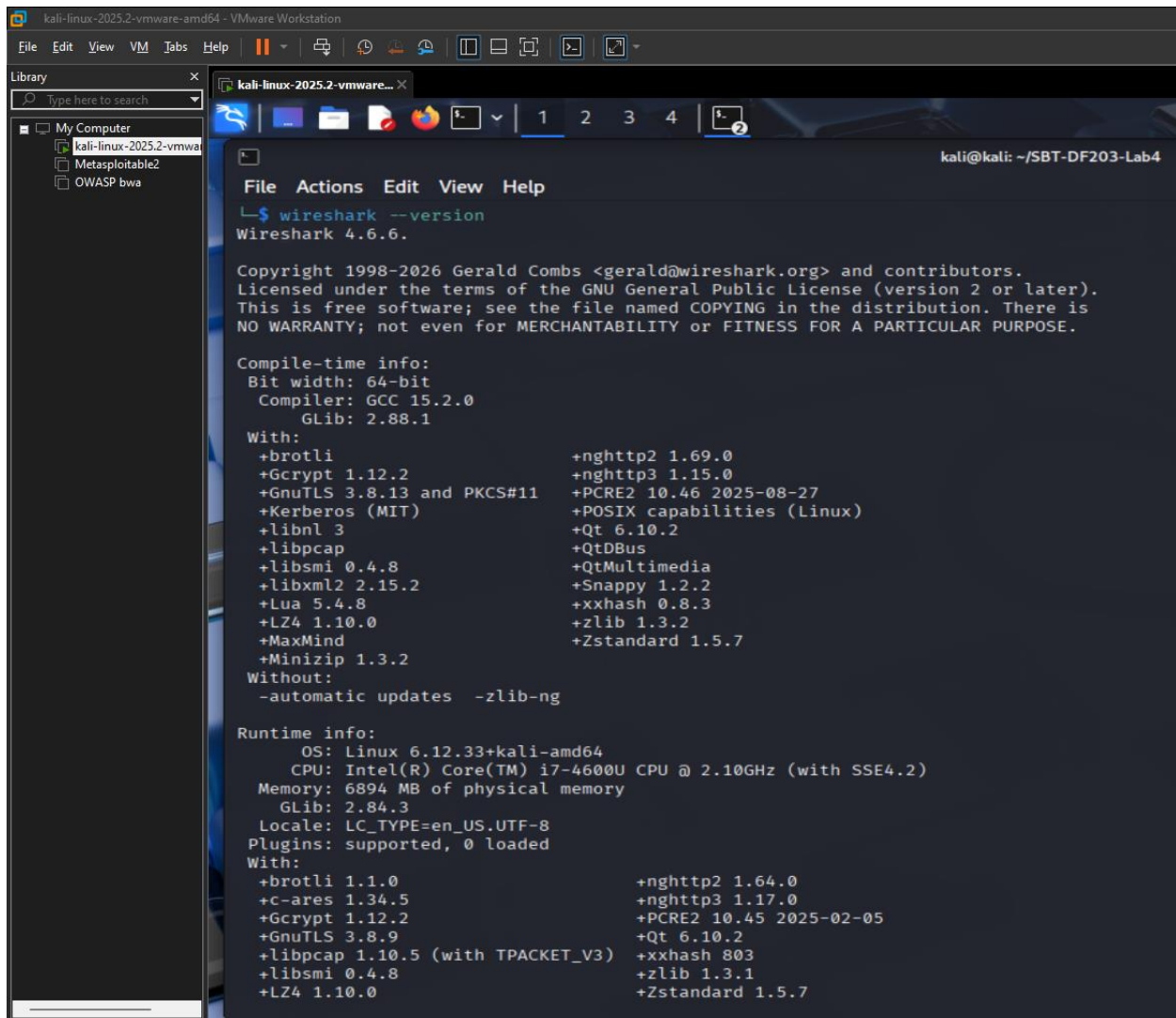
Analysis Environment and Tool Verification

Following the creation and verification of the laboratory workspace, the analysis environment was checked to confirm the availability of the tools required for the SMTP traffic investigation. Three primary tools were verified: Wireshark, TShark and Python 3. These tools support the packet-analysis, command-line examination and offline Base64 decoding activities required for the laboratory.

Wireshark

Wireshark was verified as installed and available on the Kali Linux virtual machine. Wireshark provides the graphical packet-analysis environment used to inspect the supplied SMTP packet capture, apply protocol filters, examine individual packets and reconstruct the SMTP communication using the Follow TCP Stream functionality.

The installed version was recorded as Wireshark 4.6.6



```
kali@kali: ~/SBT-DF203-Lab4

File Actions Edit View Help

L$ wireshark --version
Wireshark 4.6.6.

Copyright 1998-2026 Gerald Combs <gerald@wireshark.org> and contributors.
Licensed under the terms of the GNU General Public License (version 2 or later).
This is free software; see the file named COPYING in the distribution. There is
NO WARRANTY; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.

Compile-time info:
  Bit width: 64-bit
  Compiler: GCC 15.2.0
  GLib: 2.88.1
With:
  +brotli
  +Gcrypt 1.12.2
  +GnuTLS 3.8.13 and PKCS#11
  +Kerberos (MIT)
  +libnl 3
  +libpcap
  +libsmi 0.4.8
  +libxml2 2.15.2
  +Lua 5.4.8
  +LZ4 1.10.0
  +MaxMind
  +Minizip 1.3.2
  +nghttp2 1.69.0
  +nghttp3 1.15.0
  +PCRE2 10.46 2025-08-27
  +POSIX capabilities (Linux)
  +Qt 6.10.2
  +QtDBus
  +QtMultimedia
  +Snappy 1.2.2
  +xxhash 0.8.3
  +zlib 1.3.2
  +Zstandard 1.5.7
Without:
  -automatic updates -zlib-ng

Runtime info:
  OS: Linux 6.12.33+kali-amd64
  CPU: Intel(R) Core(TM) i7-4600U CPU @ 2.10GHz (with SSE4.2)
  Memory: 6894 MB of physical memory
  GLib: 2.84.3
  Locale: LC_TYPE=en_US.UTF-8
  Plugins: supported, 0 loaded
With:
  +brotli 1.1.0
  +c-ares 1.34.5
  +Gcrypt 1.12.2
  +GnuTLS 3.8.9
  +libpcap 1.10.5 (with TPACKET_V3)
  +libsmi 0.4.8
  +LZ4 1.10.0
  +nghttp2 1.64.0
  +nghttp3 1.17.0
  +PCRE2 10.45 2025-02-05
  +Qt 6.10.2
  +xxhash 803
  +zlib 1.3.1
  +Zstandard 1.5.7
```

TShark

TShark, the command-line network protocol analyser associated with Wireshark, was also verified. It provides a reproducible command-line method for extracting packet conversations, SMTP requests and responses, timestamps, endpoints, ports and other protocol fields from the capture. The installed version was recorded as TShark 4.6.6

The terminal displayed a libxml compatibility warning during execution; however, TShark completed execution and returned its version information successfully. The warning was therefore noted as an environment observation and did not prevent verification of the installed tool.

```
kali@kali: ~/SBT-DF203-Lab4

(kali@kali)-[~/SBT-DF203-Lab4]
$ tshark --version
Warning: program compiled against libxml 215 using older 214
TShark (Wireshark) 4.6.6.

Copyright 1998-2026 Gerald Combs <gerald@wireshark.org> and contributors.
Licensed under the terms of the GNU General Public License (version 2 or later).
This is free software; see the file named COPYING in the distribution. There is
NO WARRANTY; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.

Compile-time info:
  Bit width: 64-bit
  Compiler: GCC 15.2.0
  GLib: 2.88.1
  With:
    +brotli
    +Gcrypt 1.12.2
    +GnuTLS 3.8.13 and PKCS#11
    +Kerberos (MIT)
    +libnl 3
    +libpcap
    +libsmi 0.4.8
    +libxml2 2.15.2
    +Lua 5.4.8
    +LZ4 1.10.0
    +MaxMind
    +nghttp2 1.69.0
    +nghttp3 1.15.0
    +PCRE2 10.46 2025-08-27
    +POSIX capabilities (Linux)
    +Snappy 1.2.2
    +xxhash 0.8.3
    +zlib 1.3.2
    +Zstandard 1.5.7
  Without:
    -zlib-ng

Runtime info:
  OS: Linux 6.12.33+kali-amd64
  CPU: Intel(R) Core(TM) i7-4600U CPU @ 2.10GHz (with SSE4.2)
  Memory: 6894 MB of physical memory
  GLib: 2.84.3
  Locale: LC_TYPE=en_US.UTF-8
  Plugins: supported, 0 loaded
  With:
    +brotli 1.1.0
    +c-ares 1.34.5
    +Gcrypt 1.12.2
    +GnuTLS 3.8.9
    +libpcap 1.10.5 (with TPACKET_V3)
    +libsmi 0.4.8
    +nghttp2 1.64.0
    +nghttp3 1.17.0
    +PCRE2 10.45 2025-02-05
    +xxhash 803
    +zlib 1.3.1
    +Zstandard 1.5.7
```

Python 3

Python 3 was verified as available for the controlled offline decoding activity required later in the investigation. The laboratory specifically requires Python's base64 module for decoding selected Base64-encoded SMTP authentication values without using an online decoding service.

```
(kali@kali)-[~/SBT-DF203-Lab4]
$ python3 --version
Python 3.13.9

(kali@kali)-[~/SBT-DF203-Lab4]
$
```

Acquisition of the Supplied Packet Capture

Following verification of the analysis environment, the supplied SMTP packet capture was acquired from the authorized Wireshark training resource using the Kali Linux terminal.

The capture was downloaded directly into the laboratory's dedicated evidence directory to maintain separation between the original source evidence and subsequent analysis artefacts. The acquisition command used was `wget -O evidence/smtp.pcap 'https://wiki.wireshark.org/uploads/__moin_import__/attachments/SampleCaptures/smtp.pcap'`. The download completed successfully and the capture was saved as: `/home/kali/SBT-DF203-Lab4/evidence/smtp.pcap`

```
(kali@kali) [~/SBT-DF203-Lab4]
$ wget -O evidence/smtp.pcap 'https://wiki.wireshark.org/uploads/__moin_import__/attachments/SampleCaptures/smtp.pcap'
--2026-09-15 08:44:06-- https://wiki.wireshark.org/uploads/__moin_import__/attachments/SampleCaptures/smtp.pcap
Resolving wiki.wireshark.org (wiki.wireshark.org) ... 172.67.75.39, 104.26.11.240, 104.26.10.240, ...
Connecting to wiki.wireshark.org (wiki.wireshark.org)|172.67.75.39|:443 ... connected.
HTTP request sent, awaiting response ... 302 Found
Location: https://gitlab.com/wireshark/wireshark/-/wikis/uploads/__moin_import__/attachments/SampleCaptures/smtp.pcap [following]
--2026-09-15 08:44:09-- https://gitlab.com/wireshark/wireshark/-/wikis/uploads/__moin_import__/attachments/SampleCaptures/smtp.pcap
Resolving gitlab.com (gitlab.com) ... 172.65.251.78, 2606:4700:90:0:f22e:fbec:5bed:a9b9
Connecting to gitlab.com (gitlab.com)|172.65.251.78|:443 ... connected.
HTTP request sent, awaiting response ... 200 OK
Length: 27850 (27K) [application/octet-stream]
Saving to: 'evidence/smtp.pcap'

evidence/smtp.pcap      100%[=====] 27.20K --.-KB/s  in 0s
2026-09-15 08:44:11 (80.9 MB/s) - 'evidence/smtp.pcap' saved [27850/27850]
```

The terminal output recorded a successful HTTP response and confirmed that 27,850 bytes (27.20 KiB) were received. The downloaded file was subsequently verified at the filesystem level using `ls -lh`, which confirmed the presence of `smtp.pcap` in the designated evidence directory with a displayed size of approximately 28 KB.

```
(kali@kali) [~/SBT-DF203-Lab4]
$ wget -O evidence/smtp.pcap 'https://wiki.wireshark.org/uploads/__moin_import__/attachments/SampleCaptures/smtp.pcap'
--2026-09-15 08:44:06-- https://wiki.wireshark.org/uploads/__moin_import__/attachments/SampleCaptures/smtp.pcap
Resolving wiki.wireshark.org (wiki.wireshark.org) ... 172.67.75.39, 104.26.11.240, 104.26.10.240, ...
Connecting to wiki.wireshark.org (wiki.wireshark.org)|172.67.75.39|:443 ... connected.
HTTP request sent, awaiting response ... 302 Found
Location: https://gitlab.com/wireshark/wireshark/-/wikis/uploads/__moin_import__/attachments/SampleCaptures/smtp.pcap [following]
--2026-09-15 08:44:09-- https://gitlab.com/wireshark/wireshark/-/wikis/uploads/__moin_import__/attachments/SampleCaptures/smtp.pcap
Resolving gitlab.com (gitlab.com) ... 172.65.251.78, 2606:4700:90:0:f22e:fbec:5bed:a9b9
Connecting to gitlab.com (gitlab.com)|172.65.251.78|:443 ... connected.
HTTP request sent, awaiting response ... 200 OK
Length: 27850 (27K) [application/octet-stream]
Saving to: 'evidence/smtp.pcap'

evidence/smtp.pcap      100%[=====] 27.20K --.-KB/s  in 0s
2026-09-15 08:44:11 (80.9 MB/s) - 'evidence/smtp.pcap' saved [27850/27850]

(kali@kali) [~/SBT-DF203-Lab4]
$ ls -lh evidence/
total 28K
-rw-rw-r-- 1 kali kali 28K Sep 15 08:44 smtp.pcap
```

Creation of Working Copy

To preserve the integrity of the acquired packet capture, a separate working copy was created from the original evidence file. The original `smtp.pcap` file was retained in the evidence directory, while `smtp_working.pcap` was created in the working directory for subsequent forensic analysis. The copy was created using the `cp --preserve=timestamps` command so that the file timestamps were retained. The original evidence was therefore kept separate from the file intended for analysis.

```

(kali@kali)-[~/SBT-DF203-Lab4]
$ ls -lh evidence/
total 28K
-rw-rw-r-- 1 kali kali 28K Sep 15 08:44 smtp.pcap

(kali@kali)-[~/SBT-DF203-Lab4]
$ cp --preserve=timestamps evidence/smtp.pcap working/smtp_working.pcap

(kali@kali)-[~/SBT-DF203-Lab4]
$ cp --preserve=timestamps evidence/smtp.pcap working/smtp_working.pcap

(kali@kali)-[~/SBT-DF203-Lab4]
$

```

SHA-256 Integrity Verification

Before beginning packet-level analysis, SHA-256 hashes were calculated for both the original packet capture and the working copy. The hashes were generated using sha256sum and saved to reports/smtp_capture_hashes.txt for documentation and future verification. Both files produced the same SHA-256 hash:

17ad230db1b6fd5dd18eb311092df1cf6eb162054bdb47697b89bef5a86a47ab

```

(kali@kali)-[~/SBT-DF203-Lab4]
$ ls -lh evidence/
total 28K
-rw-rw-r-- 1 kali kali 28K Sep 15 08:44 smtp.pcap

(kali@kali)-[~/SBT-DF203-Lab4]
$ cp --preserve=timestamps evidence/smtp.pcap working/smtp_working.pcap

(kali@kali)-[~/SBT-DF203-Lab4]
$ cp --preserve=timestamps evidence/smtp.pcap working/smtp_working.pcap

(kali@kali)-[~/SBT-DF203-Lab4]
$ sha256sum evidence/smtp.pcap working/smtp_working.pcap | tee reports/smtp_capture_hashes.txt
17ad230db1b6fd5dd18eb311092df1cf6eb162054bdb47697b89bef5a86a47ab evidence/smtp.pcap
17ad230db1b6fd5dd18eb311092df1cf6eb162054bdb47697b89bef5a86a47ab working/smtp_working.pcap

(kali@kali)-[~/SBT-DF203-Lab4]
$

```

This confirms that the working copy is bit-for-bit identical to the acquired evidence at the time of copying. The original evidence/smtp.pcap can therefore be preserved while subsequent analysis is performed against working/smtp_working.pcap.

```

(kali@kali)-[~/SBT-DF203-Lab4]
$ cp --preserve=timestamps evidence/smtp.pcap working/smtp_working.pcap

(kali@kali)-[~/SBT-DF203-Lab4]
$ cp --preserve=timestamps evidence/smtp.pcap working/smtp_working.pcap

(kali@kali)-[~/SBT-DF203-Lab4]
$ sha256sum evidence/smtp.pcap working/smtp_working.pcap | tee reports/smtp_capture_hashes.txt
17ad230db1b6fd5dd18eb311092df1cf6eb162054bdb47697b89bef5a86a47ab evidence/smtp.pcap
17ad230db1b6fd5dd18eb311092df1cf6eb162054bdb47697b89bef5a86a47ab working/smtp_working.pcap

(kali@kali)-[~/SBT-DF203-Lab4]
$ cat reports/smtp_capture_hashes.txt
17ad230db1b6fd5dd18eb311092df1cf6eb162054bdb47697b89bef5a86a47ab evidence/smtp.pcap
17ad230db1b6fd5dd18eb311092df1cf6eb162054bdb47697b89bef5a86a47ab working/smtp_working.pcap

```

Capture File Characteristics

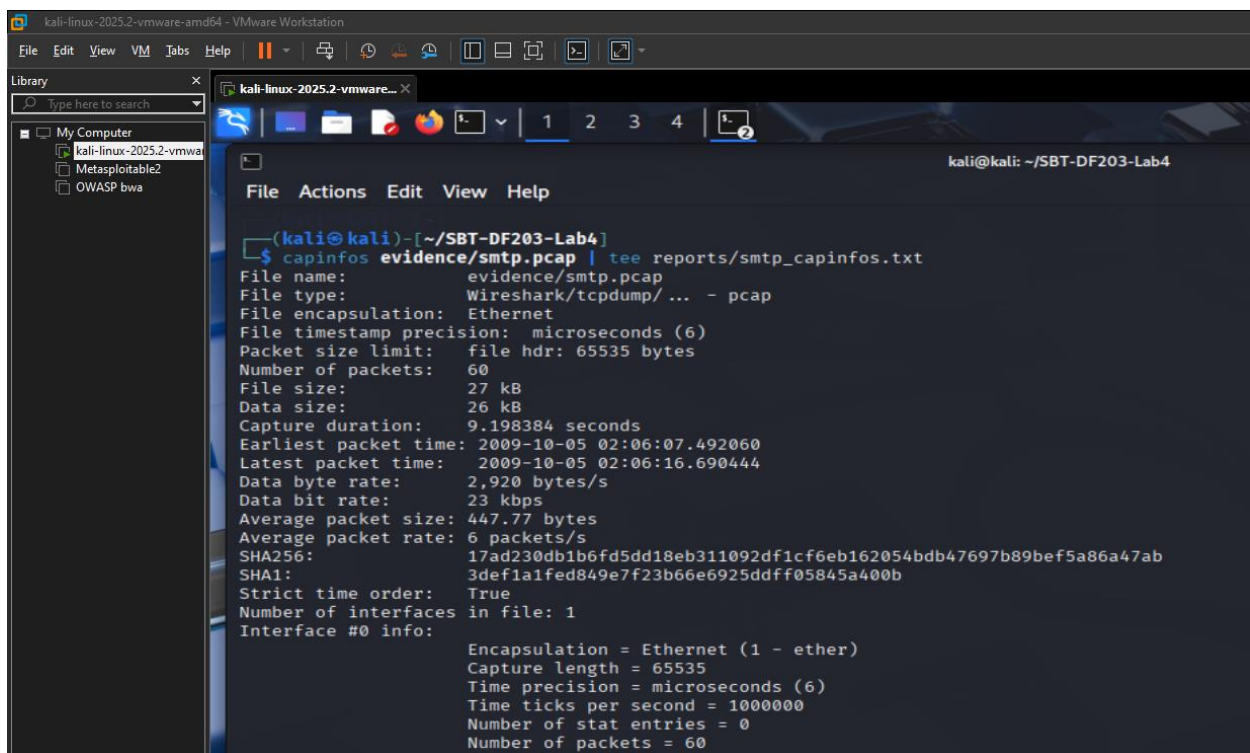
The acquired packet capture was examined using capinfos to document its basic forensic characteristics before detailed packet analysis. The capture is a PCAP file using Ethernet encapsulation and contains 60 packets with a total file size of approximately 27 kB.

The capture spans approximately 9.20 seconds, beginning at 2009-10-05 02:06:07.492060 and ending at 2009-10-05 02:06:16.690444. The capture uses microsecond timestamp precision and contains packets from one capture interface. capinfos also confirmed that the packets are in strict chronological order.

The SHA-256 value reported by capinfos matches the previously calculated SHA-256 hash for the original evidence:

17ad230db1b6fd5dd18eb311092df1cf6eb162054bdb47697b89bef5a86a47ab

This provides an additional confirmation of the recorded evidence hash before analysis begins.



```
kali@kali: ~/SBT-DF203-Lab4
File Actions Edit View Help

(kali@kali)-[~/SBT-DF203-Lab4]
$ capinfos evidence/smtp.pcap | tee reports/smtp_capinfos.txt
File name:      evidence/smtp.pcap
File type:      Wireshark/tcpdump/... - pcap
File encapsulation: Ethernet
File timestamp precision: microseconds (6)
Packet size limit: file hdr: 65535 bytes
Number of packets: 60
File size:      27 kB
Data size:      26 kB
Capture duration: 9.198384 seconds
Earliest packet time: 2009-10-05 02:06:07.492060
Latest packet time:  2009-10-05 02:06:16.690444
Data byte rate:    2,920 bytes/s
Data bit rate:     23 kbps
Average packet size: 447.77 bytes
Average packet rate: 6 packets/s
SHA256:            17ad230db1b6fd5dd18eb311092df1cf6eb162054bdb47697b89bef5a86a47ab
SHA1:              3def1a1fed849e7f23b66e6925ddff05845a400b
Strict time order: True
Number of interfaces in file: 1
Interface #0 info:
  Encapsulation = Ethernet (1 - ether)
  Capture length = 65535
  Time precision = microseconds (6)
  Time ticks per second = 1000000
  Number of stat entries = 0
  Number of packets = 60
```

Capture Metadata Recorded for Analysis

The capinfos output provides the baseline characteristics that will be used to frame the subsequent SMTP traffic analysis. In particular, the capture contains 60 packets over a

9.198384-second period, with the earliest packet recorded at 02:06:07.492060 and the latest at 02:06:16.690444 on 5 October 2009.

The capture contains 26 kB of packet data, with an average packet size of 447.77 bytes and an average packet rate of approximately 6 packets per second. The capture is also confirmed to be in strict time order, which supports chronological examination of the SMTP conversation. The capinfos results were saved to reports/smtp_capinfos.txt

```
(kali@kali)-[~/SBT-DF203-Lab4]
$ capinfos evidence/smtp.pcap | tee reports/smtp_capinfos.txt
File name:          evidence/smtp.pcap
File type:          Wireshark/tcpdump/... - pcap
File encapsulation: Ethernet
File timestamp precision: microseconds (6)
Packet size limit:  file hdr: 65535 bytes
Number of packets:  60
File size:          27 kB
Data size:          26 kB
Capture duration:   9.198384 seconds
Earliest packet time: 2009-10-05 02:06:07.492060
Latest packet time:  2009-10-05 02:06:16.690444
Data byte rate:     2,920 bytes/s
Data bit rate:      23 kbps
Average packet size: 447.77 bytes
Average packet rate: 6 packets/s
SHA256:             17ad230db1b6fd5dd18eb311092df1cf6eb162054bdb47697b89bef5a86a47ab
SHA1:               3def1a1fed849e7f23b66e6925ddff05845a400b
Strict time order:  True
Number of interfaces in file: 1
Interface #0 info:
    Encapsulation = Ethernet (1 - ether)
    Capture length = 65535
    Time precision = microseconds (6)
    Time ticks per second = 1000000
    Number of stat entries = 0
    Number of packets = 60
```

Verification of Evidence and Working Copy Integrity

As a final pre-analysis integrity check, the SHA-256 hash values of the original evidence file and the working copy were recalculated and compared. Both evidence/smtp.pcap and working/smtp_working.pcap produced the identical SHA-256 hash value:

17ad230db1b6fd5dd18eb311092df1cf6eb162054bdb47697b89bef5a86a47ab

The matching hash values confirm that the working copy remains identical to the original acquired packet capture. All subsequent packet-level analysis will therefore be performed on the verified working copy, while the original evidence remains preserved in the evidence directory.

```

(kali@kali)-[~/SBT-DF203-Lab4]
$ capinfos evidence/sntp.pcap | tee reports/sntp_capinfos.txt
File name:          evidence/sntp.pcap
File type:          Wireshark/tcpdump/... - pcap
File encapsulation: Ethernet
File timestamp precision: microseconds (6)
Packet size limit:  file hdr: 65535 bytes
Number of packets:  60
File size:          27 kB
Data size:          26 kB
Capture duration:   0.198384 seconds
Earliest packet time: 2009-10-05 02:06:07.492060
Latest packet time:  2009-10-05 02:06:16.690444
Data byte rate:     2,920 bytes/s
Data bit rate:      23 kbps
Average packet size: 447.77 bytes
Average packet rate: 6 packets/s
SHA256:             17ad230db1b6fd5dd18eb311092df1cf6eb162054bdb47697b89bef5a86a47ab
SHA1:               3def1a1fed849e7f23b66e6925ddff05845a400b
Strict time order:  True
Number of interfaces in file: 1
Interface #0 info:  Encapsulation = Ethernet (1 - ether)
                   Capture length = 65535
                   Time precision = microseconds (6)
                   Time ticks per second = 1000000
                   Number of stat entries = 0
                   Number of packets = 60

(kali@kali)-[~/SBT-DF203-Lab4]
$ sha256sum evidence/sntp.pcap working/sntp_working.pcap
17ad230db1b6fd5dd18eb311092df1cf6eb162054bdb47697b89bef5a86a47ab  evidence/sntp.pcap
17ad230db1b6fd5dd18eb311092df1cf6eb162054bdb47697b89bef5a86a47ab  working/sntp_working.pcap

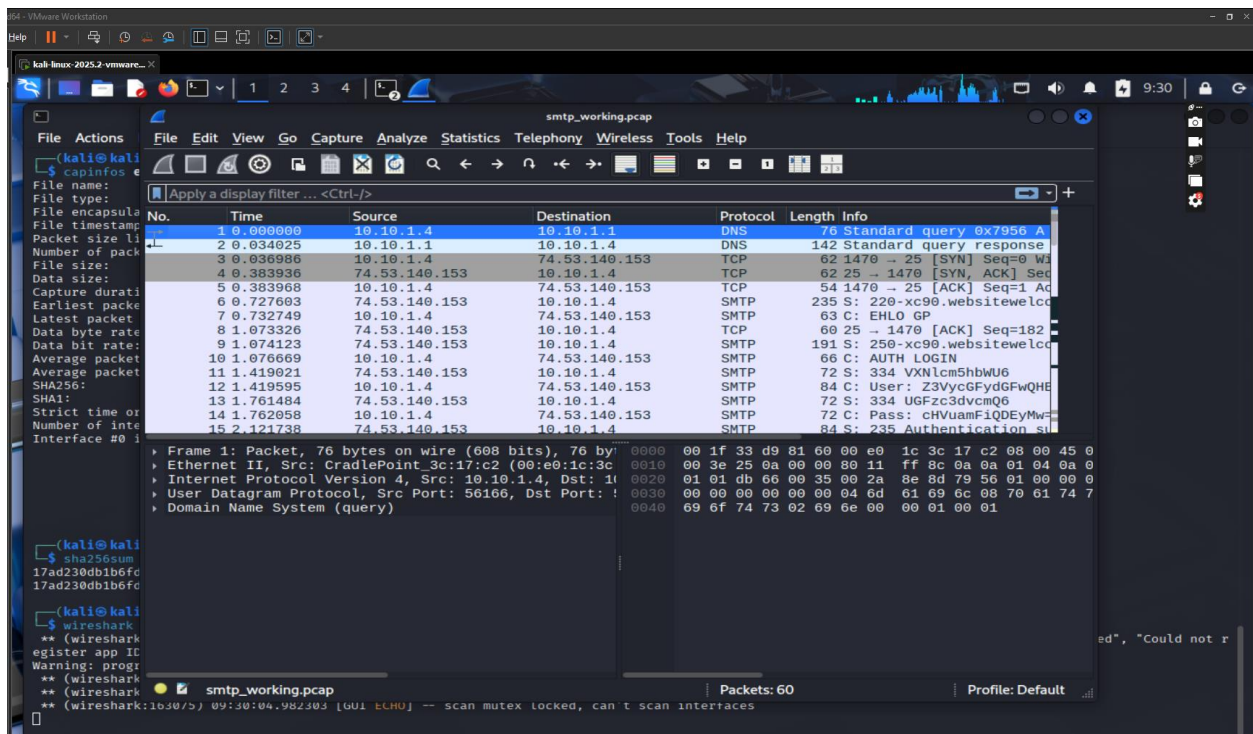
```

Opening the Verified Working Copy in Wireshark

The verified working copy of the packet capture, `sntp_working.pcap`, was opened in Wireshark for packet-level forensic analysis. The capture contains 60 packets, with the packet list showing DNS, TCP, and SMTP traffic between the identified endpoints.

The initial inspection provides an overview of the captured communication and confirms that SMTP traffic is present within the working evidence. The packet list also shows the progression from the TCP connection establishment to the SMTP conversation, including the server greeting, client EHLO command, authentication exchange, and subsequent SMTP responses.

At this stage, the capture was reviewed without modifying the underlying evidence. Detailed examination of the SMTP commands, authentication data, email content, network metadata, and encryption characteristics was performed in subsequent analysis steps.



SMTP Traffic Identification and Initial Conversation Review

To isolate the email-related traffic from the other packets in the capture, the Wireshark display filter `smtp` was applied to the verified working copy. This reduced the packet view to frames identified as Simple Mail Transfer Protocol traffic and provided a chronological view of the client-server exchange.

The filtered traffic shows communication between the SMTP server at 74.53.140.153 and the client at 10.10.1.4, with the server communicating over TCP port 25. The visible exchange begins with the SMTP server greeting and proceeds through the client EHLO command, authentication using AUTH LOGIN, and the subsequent email submission sequence.

The conversation visible in the filtered packet list includes 220, EHLO, AUTH LOGIN, username and password prompts, successful authentication (235), MAIL FROM, RCPT TO, and DATA, followed by the server's 354 response requesting the message contents. This provides the initial command sequence that will be examined in greater detail during the SMTP timeline analysis.

No.	Time	Source	Destination	Protocol	Length	Info
6	0.727603	74.53.140.153	10.10.1.4	SMTP	235	S: 220-xc90.websitewelco
7	0.732749	10.10.1.4	74.53.140.153	SMTP	63	C: EHLO GP
9	1.074123	74.53.140.153	10.10.1.4	SMTP	191	S: 250-xc90.websitewelco
10	1.076669	10.10.1.4	74.53.140.153	SMTP	66	C: AUTH LOGIN
11	1.419021	74.53.140.153	10.10.1.4	SMTP	72	S: 334 VXNlcm5hbWU6
12	1.419595	10.10.1.4	74.53.140.153	SMTP	84	C: User: Z3VycGFydGFwQHE
13	1.761484	74.53.140.153	10.10.1.4	SMTP	72	S: 334 UGFzc3dvcmQ6
14	1.762058	10.10.1.4	74.53.140.153	SMTP	72	C: Pass: cHVuamFiQDEyMw=
15	2.121738	74.53.140.153	10.10.1.4	SMTP	84	S: 235 Authentication su
16	2.122354	10.10.1.4	74.53.140.153	SMTP	90	C: MAIL FROM: <gurpartap
17	2.464705	74.53.140.153	10.10.1.4	SMTP	62	S: 250 OK
18	2.465190	10.10.1.4	74.53.140.153	SMTP	93	C: RCPT TO: <raj_deol206
19	2.827648	74.53.140.153	10.10.1.4	SMTP	68	S: 250 Accepted
20	2.828143	10.10.1.4	74.53.140.153	SMTP	60	C: DATA
21	3.169619	74.53.140.153	10.10.1.4	SMTP	110	S: 354 Enter message, en

Frame 6: Packet, 235 bytes on wire (1880 bits), 235	0000	00 e0 1c 3c 17 c2 00 1f 33 d9 81 60 08 00 45 6
▶ Ethernet II, Src: Netgear_d9:81:60 (00:1f:33:d9:81:60), Dst: 10.10.1.4 (08:00:00:08:00:04)	0010	00 dd 21 e2 40 00 32 06 43 fd 4a 35 8c 99 0a 0
▶ Internet Protocol Version 4, Src: 74.53.140.153, Dst: 10.10.1.4	0020	01 04 00 19 05 be ae ec 61 b0 7e c4 53 b1 50 1
▶ Transmission Control Protocol, Src Port: 25, Dst Port: 1470	0030	16 d0 8a 44 00 00 32 32 30 2d 78 63 39 30 2e 7
▶ Simple Mail Transfer Protocol	0040	65 62 73 69 74 65 77 65 6c 63 6f 6d 65 2e 63 6
	0050	6d 20 45 53 4d 54 50 20 45 78 69 6d 20 34 2e 3
	0060	39 20 23 31 20 4d 6f 6e 2c 20 30 35 20 4f 63 7
	0070	20 32 30 30 39 20 30 31 3a 30 35 3a 35 34 20 2
	0080	30 35 30 30 20 0d 0a 32 32 30 2d 57 65 20 64 6
	0090	20 6e 6f 74 20 61 75 74 68 6f 72 69 7a 65 20 7
	00a0	68 65 20 75 73 65 20 6f 66 20 74 68 69 73 20 7
	00b0	79 73 74 65 6d 20 74 6f 20 74 72 61 6e 73 70 6
	00c0	72 74 20 75 6e 73 6f 6c 69 63 69 74 65 64 2c 2
	00d0	0d 0a 32 32 30 20 61 6e 64 2f 6f 72 20 62 75 6
	00e0	6b 20 65 2d 6d 61 69 6c 2e 0d 0a

TCP Conversation Inventory

Before extracting individual SMTP commands and responses, the TCP conversations within the working packet capture were examined using TShark. The analysis identified a single TCP conversation associated with the SMTP session.

The conversation occurred between the client 10.10.1.4 using ephemeral port 1470 and the SMTP server 74.53.140.153 using TCP port 25. A total of 53 TCP frames were associated with this conversation, consisting of 25 frames from the server toward the client and 28 frames from the client toward the server. The conversation transferred approximately 24 kB of data and lasted approximately 7.58 seconds.

The relative start time reported by TShark was 0.036986 seconds from the beginning of the capture. This establishes the primary TCP session to be examined for the subsequent SMTP command, authentication, email-content, and network metadata analysis.

```

kali@kali: ~/SBT-DF203-Lab4
$ cd ~/SBT-DF203-Lab4
$ tshark -r working/smtworking.pcap -d -z conv,tcp
Warning: program compiled against libxml 215 using older 214

TCP Conversations
Filter:<No Filter>

| Frames | Bytes | | Frames | Bytes | | Total | Relative | Duration |
|-----|-----| |-----|-----| |-----| Start    |          |
10.10.1.4:1470 <--> 74.53.140.153:25 | 25 | 1,980 bytes | 28 | 22 kB | 53 | 24 kB | 0.036986000 | 7.5777

```

What we have established so far include the following findings:

Item	Finding
SMTP client	10.10.1.4
Client port	1470
SMTP server	74.53.140.153
Server port	25
Server → Client	25 frames / 1,980 bytes
Client → Server	28 frames / ~22 kB
Total	53 frames / ~24 kB
Relative start	0.036986 seconds
Duration	7.5777 seconds

SMTP Command and Response Inventory

A detailed SMTP packet inventory was extracted from the verified working capture using TShark. The extraction records the frame number, relative timestamp, source and destination addresses, TCP ports, SMTP request commands and parameters, and server response codes and parameters. The SMTP session begins with the server's 220 greeting in frame 6, followed by the client's EHLO command in frame 7. The server responds with 250 and advertises several capabilities, including AUTH PLAIN LOGIN and STARTTLS. The client subsequently issues AUTH LOGIN, resulting in the expected 334 authentication prompts and a 235 Authentication succeeded response.

Following successful authentication, the client submits the message using MAIL FROM and RCPT TO. Both commands receive successful 250 responses. The client then issues

DATA, and the server responds with 354, indicating that the message content can be transmitted. Multiple subsequent frames carry the message data before the server returns a 250 OK response with a message identifier. The session concludes with the client issuing QUIT in frame 54 and the server returning 221, indicating that the SMTP connection is being closed.

```

(kali@kali)~/SBT-DF203-Lab4
$ tshark -r working/smtp_working.pcap -Y 'smtp' \
  -o frame.number \
  -o frame.time_relative \
  -o ip.src \
  -o ip.dst \
  -o tcp.srcport \
  -o tcp.dstport \
  -o smtp.req.command \
  -o smtp.req.parameter \
  -o smtp.response.code \
  -o smtp.rsp.parameter
Warning: program compiled against libm 215 using older 214
6 0.727603000 74.53.140.153 10.10.1.4 25 1470 xc90.websitewelcome.com ESMTPEXIM 4.69 #1 Mon, 05 Oct 200
9 01:05:54 -0500 We do not authorize the use of this system to transport unsolicited,
and/or bulk e-mail.
7 0.732749000 10.10.1.4 74.53.140.153 1470 25 EHLO GP 250 xc90.websitewelcome.com Hello GP [122.162.143.157],SIZE 52
428800 PIPELINING, AUTH PLAIN LOGIN, STARTTLS, HELP
10 1.076669000 10.10.1.4 74.53.140.153 1470 25 AUTH LOGIN 334 VXNlcm5hbWU6
11 1.419021000 74.53.140.153 10.10.1.4 25 1470 334 UGFzc3dvcmQ6
12 1.419595000 10.10.1.4 74.53.140.153 1470 25 235 Authentication succeeded
13 1.761664000 74.53.140.153 10.10.1.4 25 1470 FROM: <gurpartapatriots.in>
14 1.762058000 10.10.1.4 74.53.140.153 1470 25 250 OK
15 2.121738000 74.53.140.153 10.10.1.4 25 1470 RCPT TO: <raj_deol20021n@yahoo.co.in>
16 2.122354000 10.10.1.4 74.53.140.153 1470 25 250 Accepted
17 2.464705000 74.53.140.153 10.10.1.4 25 1470 DATA 354 Enter message, ending with "." on a line by itself
18 2.465190000 10.10.1.4 74.53.140.153 1470 25
19 2.827648000 74.53.140.153 10.10.1.4 25 1470
20 2.828143000 10.10.1.4 74.53.140.153 1470 25
21 3.109619000 74.53.140.153 10.10.1.4 25 1470
22 3.200683000 10.10.1.4 74.53.140.153 1470 25
23 3.200726000 10.10.1.4 74.53.140.153 1470 25
24 3.200744000 10.10.1.4 74.53.140.153 1470 25
25 3.200763000 10.10.1.4 74.53.140.153 1470 25
26 3.203055000 192.168.1.1,10.10.1.4 10.10.1.4,74.53.140.153 1470 25
27 3.203563000 192.168.1.1,10.10.1.4 10.10.1.4,74.53.140.153 1470 25
28 3.204180000 192.168.1.1,10.10.1.4 10.10.1.4,74.53.140.153 1470 25
29 3.204574000 192.168.1.1,10.10.1.4 10.10.1.4,74.53.140.153 1470 25
30 3.612912000 10.10.1.4 74.53.140.153 1470 25
31 3.612938000 10.10.1.4 74.53.140.153 1470 25
32 3.977699000 10.10.1.4 74.53.140.153 1470 25
33 3.977754000 10.10.1.4 74.53.140.153 1470 25
34 4.002121000 10.10.1.4 74.53.140.153 1470 25
35 4.002139000 10.10.1.4 74.53.140.153 1470 25
36 4.342568000 10.10.1.4 74.53.140.153 1470 25
37 4.342595000 10.10.1.4 74.53.140.153 1470 25
38 4.366256000 10.10.1.4 74.53.140.153 1470 25
39 4.366274000 10.10.1.4 74.53.140.153 1470 25
40 4.756729000 74.53.140.153 10.10.1.4 25 1470
41 7.271765000 10.10.1.4 74.53.140.153 1470 25 QUIT
42 7.613407000 74.53.140.153 10.10.1.4 25 1470 221 xc90.websitewelcome.com closing connection

```

The important timeline we have now

Frame	Time (s)	Direction	SMTP activity
6	0.727603	Server → Client	220 server greeting
7	0.732749	Client → Server	EHLO
9	1.074123	Server → Client	250 capabilities
10	1.076669	Client → Server	AUTH LOGIN
11	1.419021	Server → Client	334 username prompt

Frame	Time (s)	Direction	SMTP activity
12	1.419595	Client → Server	Base64 authentication value
13	1.761484	Server → Client	334 password prompt
14	1.762058	Client → Server	Base64 authentication value
15	2.121738	Server → Client	235 Authentication succeeded
16	2.122354	Client → Server	MAIL FROM
17	2.464705	Server → Client	250 OK
18	2.465190	Client → Server	RCPT TO
19	2.827648	Server → Client	250 Accepted
20	2.828143	Client → Server	DATA
21	3.169619	Server → Client	354 Enter message...
22–45	~3.20–4.37	Client → Server	Message data
52	4.756729	Server → Client	250 OK, message accepted
54	7.271765	Client → Server	QUIT
56	7.613407	Server → Client	221 connection closing

One important forensic observation is the EHLO response in frame 9 is particularly important because the server advertises “AUTH PLAIN LOGIN, STARTTLS”. This means the server supported STARTTLS, but that alone does not mean this particular SMTP session was encrypted. We'll formally establish that in the encryption section later.

Also, frames 12 and 14 contain the Base64-encoded authentication values. Do not decode or put those credentials into the report yet. That's our next dedicated section, where we'll demonstrate the Base64 issue while masking the sensitive values.

SMTP Authentication and Base64-Encoded Credentials

The SMTP authentication exchange was examined using TShark to identify authentication-related fields within the packet capture. Two client-to-server authentication values were identified in frames 12 and 14. The values were transmitted as Base64-encoded strings during the AUTH LOGIN exchange.

Frame 12 contains the Base64-encoded username, while frame 14 contains the Base64-encoded password. The server subsequently returned 235 Authentication succeeded in frame 15, confirming that the authentication exchange was successful.

For security and privacy purposes, the decoded credentials are not reproduced in this report. The authentication values are treated as sensitive evidence and are masked wherever they are referenced in the report.

Extraction of Base64-Encoded SMTP Authentication Values

The screenshot shows the TShark extraction of authentication-related SMTP fields. Frame 12 contains the Base64-encoded username and frame 14 contains the Base64-encoded password. The values demonstrate that the SMTP authentication data was transmitted in an encoded rather than encrypted form.

What your output tells us

Frame	Time	Direction	Finding
12	1.419595 s	Client → Server	Base64 username
14	1.762058 s	Client → Server	Base64 password
15	2.121738 s	Server → Client	235 Authentication succeeded

The actual values you obtained are:

- Frame 12: Z3VycGFydGFwQHBhdHJpb3RzLmlu
- Frame 14: cHVuamFiQDEyMw==

```
(kali@kali)~[~/SBT-DF203-Lab4]
$ tshark -r working/smtp_working.pcap -Y 'smtp.auth.username || smtp.auth.password || smtp.auth.username_password' \
-T fields \
-e frame.number \
-e frame.time_relative \
-e ip.src \
-e ip.dst \
-e smtp.auth.username \
-e smtp.auth.password \
-e smtp.auth.username_password
Warning: program compiled against libxml 215 using older 214
12  1.419595000    10.10.1.4      74.53.140.153  Z3VycGFydGFwQHBhdHJpb3RzLmlu
14  1.762058000    10.10.1.4      74.53.140.153  cHVuamFiQDEyMw==
```

Offline Base64 Decoding and Credential Recovery

The Base64-encoded authentication values identified in frames 12 and 14 were decoded offline using Python's standard base64 module. The decoding successfully recovered the original SMTP authentication credentials from the packet capture. The username recovered from frame 12 was "gurpartap@patriots.in". The password recovered from frame 14 was "punjab@123".

```
(kali@kali)-[~/SBT-DF203-Lab4]
$ python3 - <<'PY'
import base64

values = [
    "Z3VycGFydGFwQHBhdHJpb3RzLmlu",
    "cHVuamFiQDEyMw=="
]

for value in values:
    print(base64.b64decode(value).decode())
PY
gurpartap@patriots.in
punjab@123
```

The server subsequently returned 235 Authentication succeeded in frame 15, confirming that the recovered credentials were accepted during the captured SMTP session.

This demonstrates that Base64 encoding does not provide confidentiality. Because the SMTP authentication exchange was transmitted without an established TLS session, an investigator with access to the packet capture could recover the credentials by extracting and decoding the Base64 values.

SMTP Conversation Reconstruction

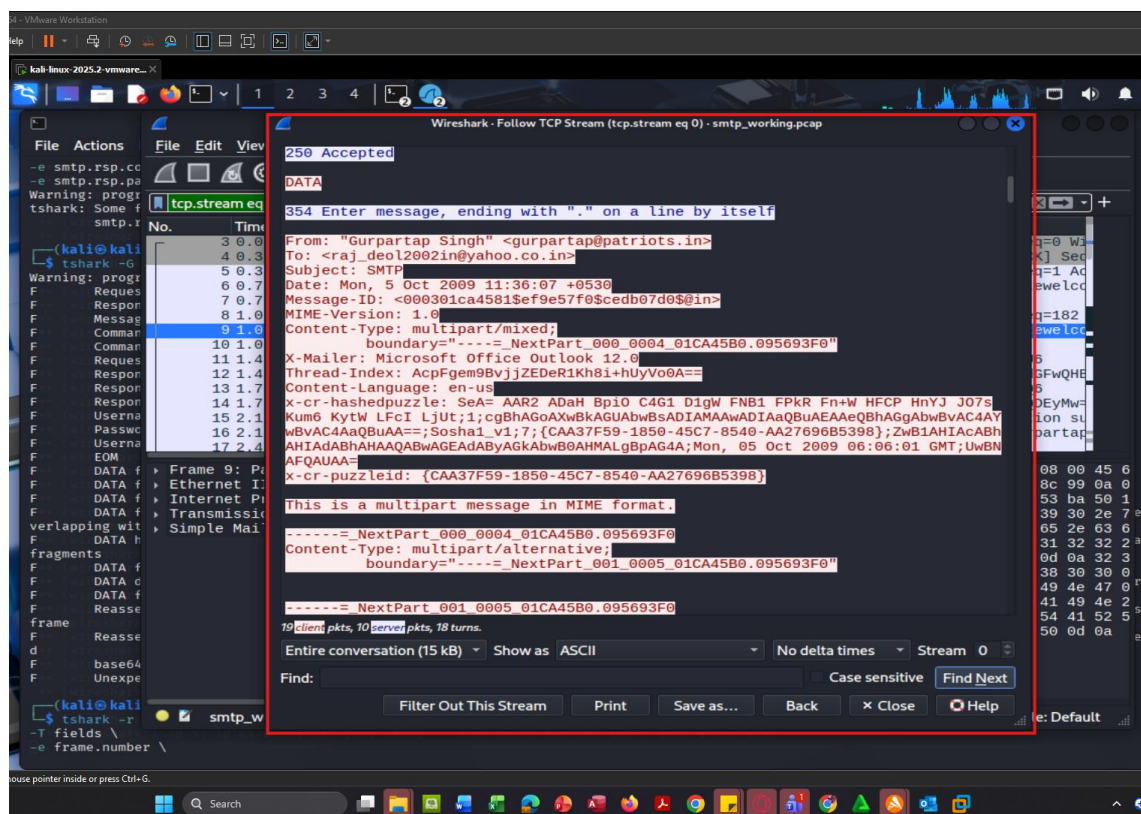
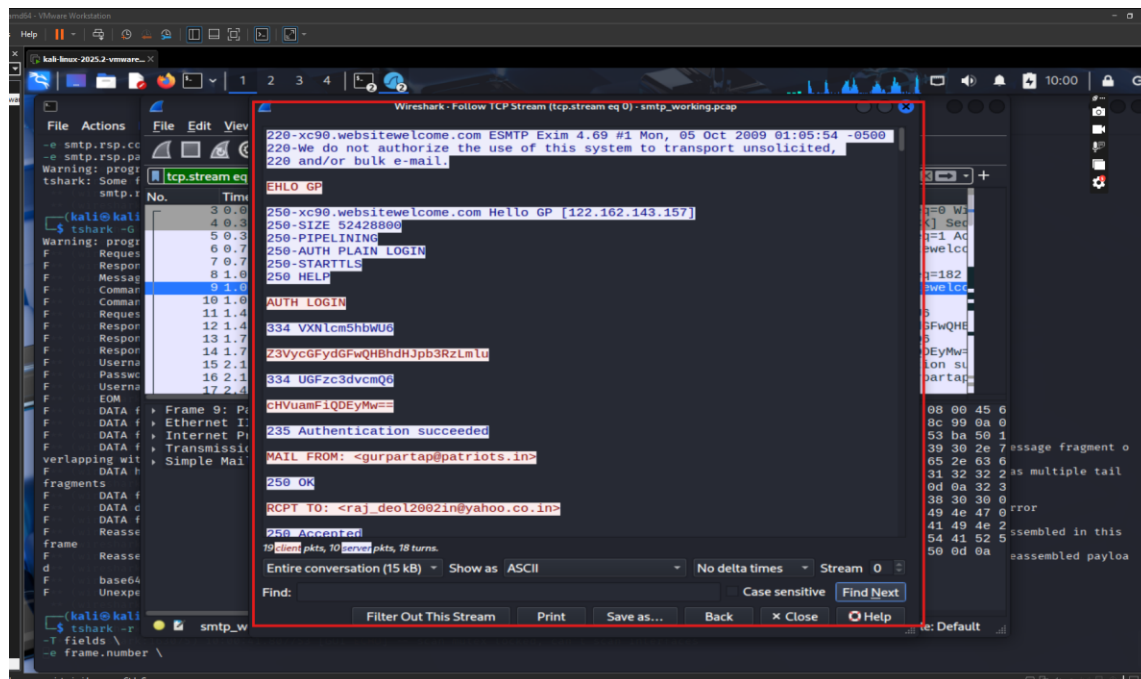
The SMTP session was reconstructed from the packet capture to examine the complete sequence of client-server communication and the email message transmitted during the session. The reconstructed conversation begins with the SMTP server's 220 greeting and proceeds through EHLO, authentication using AUTH LOGIN, successful authentication, message envelope commands, message transmission, and session termination.

The server's EHLO response identifies the SMTP service as Exim 4.69 and advertises capabilities including AUTH PLAIN LOGIN and STARTTLS. The client subsequently authenticates using the AUTH LOGIN mechanism. The authentication exchange contains Base64-encoded values, demonstrating that the credentials were transmitted in an encoding that can be reversed when the session is not protected by TLS.

Following successful authentication, the client submits an email using MAIL FROM and RCPT TO, after which the server accepts the DATA command and receives the message content. The reconstructed message contains standard email headers including From, To, Subject, Date, Message-ID, MIME-Version, and Content-Type.

The message identifies Microsoft Office Outlook 12.0 through the X-Mailer header and uses a multipart MIME structure containing both plain-text and HTML representations of the message. The message also contains an attachment identified as NEWS.txt. The

SMTP server subsequently returns a 250 OK response with a message identifier, indicating successful acceptance of the submitted message, before the client issues QUIT and the server responds with 221.



Wireshark · Follow TCP Stream (tcp.stream eq 0) · smtp_working.pcap

```
-----=_NextPart_001_0005_01CA45B0.095693F0
Content-Type: text/plain;
    charset="us-ascii"
Content-Transfer-Encoding: 7bit

Hello

I send u smtp pcap file

Find the attachment

GPS

-----=_NextPart_001_0005_01CA45B0.095693F0
Content-Type: text/html;
    charset="us-ascii"
Content-Transfer-Encoding: quoted-printable

<html xmlns:v=3D"urn:schemas-microsoft-com:vml" =
xmlns:o=3D"urn:schemas-microsoft-com:office:office" =
xmlns:w=3D"urn:schemas-microsoft-com:office:word" =
xmlns:m=3D"http://schemas.microsoft.com/office/2004/12/omml" =
xmlns=3D"http://www.w3.org/TR/REC-html40">

<head>
<META HTTP-EQUIV=3D"Content-Type" CONTENT=3D"text/html; =
charset=3Dus-ascii">
```

19 client pkts, 10 server pkts, 18 turns.

Entire conversation (15 kB) Show as ASCII No delta times Stream 0

Find: Case sensitive Find Next

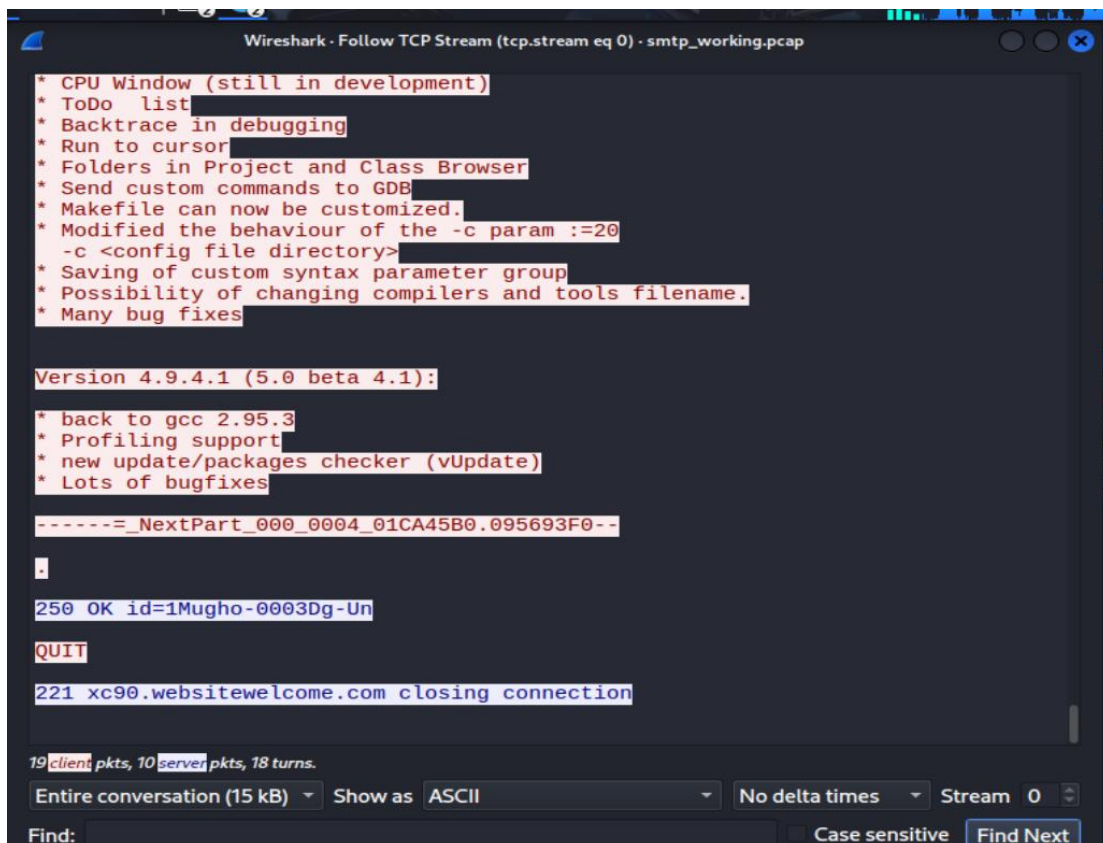
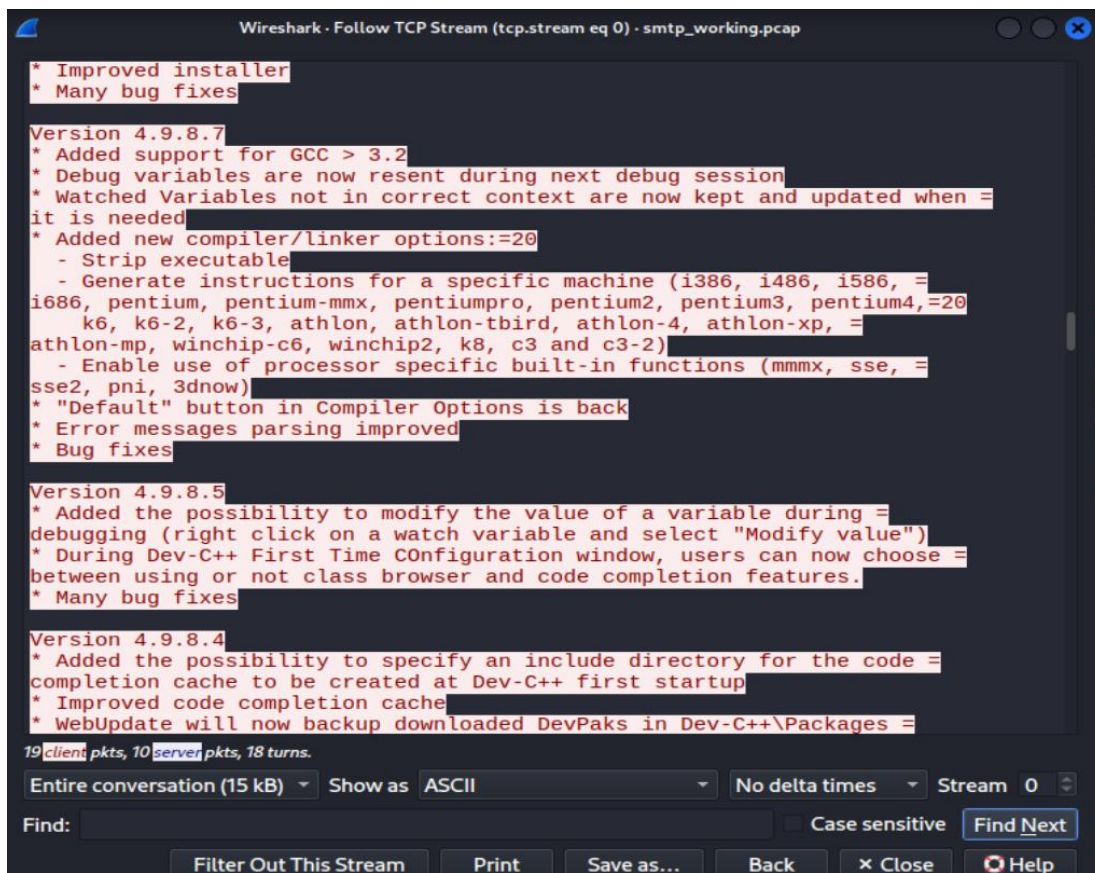
Wireshark · Follow TCP Stream (tcp.stream eq 0) · smtp_working.pcap

```
charset=3Dus-ascii">
<meta name=3DGenerator content=3D"Microsoft Word 12 (filtered medium)">
<style>
<!--
/* Font Definitions */
@font-face
    {font-family:"Cambria Math";
    panose-1:2 4 5 3 5 4 6 3 2 4;}
@font-face
    {font-family:Calibri;
    panose-1:2 15 5 2 2 2 4 3 2 4;}
/* Style Definitions */
p.MsoNormal, li.MsoNormal, div.MsoNormal
    {margin:0in;
    margin-bottom:.0001pt;
    font-size:11.0pt;
    font-family:"Calibri", "sans-serif";}
a:link, span.MsoHyperlink
    {mso-style-priority:99;
    color:blue;
    text-decoration:underline;}
a:visited, span.MsoHyperlinkFollowed
    {mso-style-priority:99;
    color:purple;
    text-decoration:underline;}
span.EmailStyle17
    {mso-style-type:personal-compose;
    font-family:"Calibri", "sans-serif";
    color:windowtext;}
..MsoChpDefault
    {mso-style-type:export-only;}
@page Section1
```

19 client pkts, 10 server pkts, 18 turns.

Entire conversation (15 kB) Show as ASCII No delta times Stream 0

Find: Case sensitive Find Next



What we can extract from this evidence include:

Evidence	Finding
SMTP software	Exim 4.69
Authentication mechanism	AUTH LOGIN
TLS capability	STARTTLS advertised
Authentication result	235 Authentication succeeded
Sender	gurpartap@patriots.in
Recipient	raj_deol2002in@yahoo.co.in
Subject	SMTP
Client software	Microsoft Office Outlook 12.0
MIME	multipart/mixed
Message body	Plain-text and HTML parts
Attachment	NEWS.txt
Attachment content	Dev-C++ version/change log
Message acceptance	250 OK
Session termination	QUIT → 221

SMTP TCP Stream Reconstruction

Following the identification of the SMTP DATA command, the TCP session was reconstructed using Wireshark's Follow TCP Stream functionality. This provided a continuous view of the SMTP client-server exchange and allowed the email transaction to be examined beyond the individual packets. The reconstructed stream was reviewed to identify the email headers, MIME information, mail client indicators, and message content contained within the captured SMTP session.

```
(kali㉿kali) [~/SBT-DF203-1b4]
$ tshark -r working/smtp_working.pcap -Y 'smtp.auth.username || smtp.auth.password || smtp.auth.username_password' \
-T fields \
-e frame.number \
-e frame.time_relative \
-e ip.src \
-e ip.dst \
-e smtp.auth.username \
-e smtp.auth.password \
-e smtp.auth.username_password
Warning: program compiled against libxml 215 using older 214
12      1.419595000      10.10.1.4      74.53.140.153      Z3VycGFyZGFwQHBhdHJpb3RzLmLu
14      1.762058000      10.10.1.4      74.53.140.153      cHVuamFicUdEYmw=
```

SMTP TCP Stream Identification

To establish the TCP stream containing the SMTP session, TShark was used to associate SMTP packets with their corresponding TCP stream identifier. The extraction showed that the SMTP conversation, beginning with the server greeting and continuing through authentication, message submission, and connection termination, occurred within TCP stream 0. This stream was therefore selected for subsequent email reconstruction and content analysis.

The screenshot shows a Kali Linux terminal window with the title bar "kali-linux-2025.2-vmware...". The terminal prompt is "kali@kali: ~/SBT-DF203-Lab4". The user has run the command `tshark -r working/smtp_working.pcap -Y 'smtp' \`. The terminal output shows the first 56 lines of the capture, which correspond to the SMTP session. A red box highlights the first 56 lines of the capture, which correspond to the SMTP session. The terminal output shows the SMTP protocol exchange between a client and a server, including commands like EHLO, AUTH, MAIL, RCPT, DATA, and QUIT. The terminal output is as follows:

```

(kali@kali)-[~/SBT-DF203-Lab4]
$ tshark -r working/smtp_working.pcap -Y 'smtp' \
-T fields \
-e frame.number \
-e tcp.stream \
-e smtp.req.command \
-e smtp.response.code
Warning: program compiled against libxml 215 using older 214
 6    0          220
 7    0      EHLO
 9    0          250
10   0      AUTH
11   0          334
12   0
13   0          334
14   0
15   0          235
16   0      MAIL
17   0          250
18   0      RCPT
19   0          250
20   0      DATA
21   0          354
22   0
23   0
24   0
25   0
26   0
27   0
28   0
29   0
30   0
31   0
32   0
33   0
34   0
35   0
36   0
37   0
38   0
39   0
40   0
41   0
42   0
43   0
44   0
45   0
46   0
47   0
48   0
49   0
50   0
51   0
52   0          250
53   0
54   0      QUIT
55   0
56   0          221

```

SMTP TCP Stream Packet Extraction

The identified TCP stream was subsequently examined using TShark to enumerate all packets belonging to the SMTP session. The output confirms that TCP stream 0 contains the complete exchange, from the initial TCP connection and SMTP greeting through authentication, MAIL FROM, RCPT TO, DATA, message transmission, QUIT, and server connection closure. The smtp.message field did not display the message content in this extraction, so the individual packet sequence was retained as the basis for further reconstruction using Wireshark's Follow TCP Stream functionality.

```
(kali@kali)~[~/SBT-DF203-Lab4]
$ tshark -r working/smtp_working.pcap \
-Y 'tcp.stream == 0' \
-T fields \
-e frame.number \
-e frame.time_relative \
-e smtp.message
Warning: program compiled against libxml 215 using older 214
3      0.036986000 10.30.75.1:54321->10.30.75.1:25 [G] 250
4      0.383968000 10.30.75.1:54321->10.30.75.1:25 [G]
5      0.383968000 10.30.75.1:54321->10.30.75.1:25 [G]
6      0.727603000 10.30.75.1:54321->10.30.75.1:25 [G]
7      0.732749000 10.30.75.1:54321->10.30.75.1:25 [G]
8      1.073326000 10.30.75.1:54321->10.30.75.1:25 [G]
9      1.074123000 10.30.75.1:54321->10.30.75.1:25 [G]
10     1.076669000 10.30.75.1:54321->10.30.75.1:25 [G]
11     1.419021000 10.30.75.1:54321->10.30.75.1:25 [G]
12     1.419595000 10.30.75.1:54321->10.30.75.1:25 [G]
13     1.761484000 10.30.75.1:54321->10.30.75.1:25 [G]
14     1.762058000 10.30.75.1:54321->10.30.75.1:25 [G]
15     2.121738000 10.30.75.1:54321->10.30.75.1:25 [G]
16     2.122354000 10.30.75.1:54321->10.30.75.1:25 [G]
17     2.464705000 10.30.75.1:54321->10.30.75.1:25 [G]
18     2.465190000 10.30.75.1:54321->10.30.75.1:25 [G]
19     2.827648000 10.30.75.1:54321->10.30.75.1:25 [G]
20     2.828143000 10.30.75.1:54321->10.30.75.1:25 [G]
21     3.169619000 10.30.75.1:54321->10.30.75.1:25 [G]
22     3.200663000 10.30.75.1:54321->10.30.75.1:25 [G]
23     3.200726000 10.30.75.1:54321->10.30.75.1:25 [G]
24     3.200744000 10.30.75.1:54321->10.30.75.1:25 [G]
25     3.200763000 10.30.75.1:54321->10.30.75.1:25 [G]
26     3.203055000 10.30.75.1:54321->10.30.75.1:25 [G]
27     3.203110000 10.30.75.1:54321->10.30.75.1:25 [G]
28     3.203563000 10.30.75.1:54321->10.30.75.1:25 [G]
29     3.204188000 10.30.75.1:54321->10.30.75.1:25 [G]
30     3.204574000 10.30.75.1:54321->10.30.75.1:25 [G]
31     3.612881000 10.30.75.1:54321->10.30.75.1:25 [G]
32     3.612912000 10.30.75.1:54321->10.30.75.1:25 [G]
33     3.612938000 10.30.75.1:54321->10.30.75.1:25 [G]
34     3.977663000 10.30.75.1:54321->10.30.75.1:25 [G]
35     3.977699000 10.30.75.1:54321->10.30.75.1:25 [G]
36     3.977754000 10.30.75.1:54321->10.30.75.1:25 [G]
37     4.002107000 10.30.75.1:54321->10.30.75.1:25 [G]
38     4.002121000 10.30.75.1:54321->10.30.75.1:25 [G]
39     4.002139000 10.30.75.1:54321->10.30.75.1:25 [G]
40     4.342535000 10.30.75.1:54321->10.30.75.1:25 [G]
41     4.342568000 10.30.75.1:54321->10.30.75.1:25 [G]
42     4.342595000 10.30.75.1:54321->10.30.75.1:25 [G]
43     4.366241000 10.30.75.1:54321->10.30.75.1:25 [G]
44     4.366256000 10.30.75.1:54321->10.30.75.1:25 [G]
45     4.366274000 10.30.75.1:54321->10.30.75.1:25 [G]
46     4.389163000 10.30.75.1:54321->10.30.75.1:25 [G]
47     4.413523000 10.30.75.1:54321->10.30.75.1:25 [G]
48     4.708119000 10.30.75.1:54321->10.30.75.1:25 [G]
49     4.730686000 10.30.75.1:54321->10.30.75.1:25 [G]
50     4.754784000 10.30.75.1:54321->10.30.75.1:25 [G]
51     4.756231000 10.30.75.1:54321->10.30.75.1:25 [G]
52     4.756729000 10.30.75.1:54321->10.30.75.1:25 [G]
53     4.895535000 10.30.75.1:54321->10.30.75.1:25 [G]
54     7.271765000 10.30.75.1:54321->10.30.75.1:25 [G]
55     7.272516000 10.30.75.1:54321->10.30.75.1:25 [G]
56     7.613407000 10.30.75.1:54321->10.30.75.1:25 [G]
57     7.613939000 10.30.75.1:54321->10.30.75.1:25 [G]
58     7.613955000 10.30.75.1:54321->10.30.75.1:25 [G]
59     7.614699000 10.30.75.1:54321->10.30.75.1:25 [G]
```

SMTP Message and DATA Reassembly Field Verification

To determine the available SMTP protocol fields for automated extraction, the TShark field registry was queried for message, authentication, and DATA-related fields. The output confirmed the availability of smtp.message and several DATA fragmentation and reassembly fields, including smtp.data.fragments, smtp.data.fragment, and smtp.data.reassembled.length. However, dedicated SMTP fields for individual email headers such as Subject, From, To, and Date were not available in the displayed field set. Consequently, the email headers and message content were examined through TCP stream reconstruction rather than relying on individual SMTP header fields.

```
(kali@kali) ~/SBT-DF203-Lab4
$ tshark -G fields | grep -E '^f.smtp\.' | grep -E 'message|data|header|subject|from|to|date|mime|user|mail'
Warning: program compiled against libxml 215 using older 214
F Message smtp.message FT_STRING smtp 0x0 smtp 0x0
F Username smtp.auth.username FT_STRING smtp 0x0
F Username/Password smtp.auth.username_password FT_STRING smtp 0x0
F DATA fragments smtp.data.fragments FT_NONE smtp 0x0 Message fragments
F DATA fragment smtp.data.fragment FT_FRAMEENUM smtp 0x0 Message fragment
F DATA fragment overlap smtp.data.fragment.overlap FT_BOOLEAN smtp 0 0x0 Message fragment overlap
F DATA fragment overlapping with conflicting data smtp.data.fragment.overlap.conflicts FT_BOOLEAN smtp 0 0x0 Message fragment overlapping with conflicting data
F DATA has multiple tail fragments smtp.data.fragment.multiple_tails FT_BOOLEAN smtp 0 0x0 Message has multiple tail fragments
F DATA fragment too long smtp.data.fragment.too_long FT_BOOLEAN smtp 0 0x0 Message fragment too long
F DATA defragmentation error smtp.data.fragment.error FT_FRAMEENUM smtp 0x0 Message defragmentation error
F DATA fragment count smtp.data.fragment.count FT_UINT32 smtp BASE_DEC 0x0
F Reassembled DATA in frame smtp.data.reassembled.in FT_FRAMEENUM smtp 0x0 This DATA fragment is reassembled in this frame
F Reassembled DATA length smtp.data.reassembled.length FT_UINT32 smtp BASE_DEC 0x0 The total length of the reassembled payload
(kali@kali) ~/SBT-DF203-Lab4
```

SMTP TCP Payload and Message-Data Segmentation

The TCP payloads within stream 0 were extracted to examine how the SMTP session was transported at the packet level. The output shows the SMTP commands and responses as individual TCP payloads, while the email submitted after the DATA command was divided across multiple TCP segments. Frames 22 through 45 contain the larger client-to-server payload segments associated with the transmitted message. This confirms that the email content requires TCP stream reassembly for complete reconstruction rather than analysis of a single packet.

```
(kali@kali)-[~/SBT-DF203-Lab4]
$ tshark -r working/smtp_working.pcap \
-Y 'tcp.stream == 0 && tcp.len > 0' \
-T fields \
-e frame.number \
-e frame.time_relative \
-e ip.src \
-e ip.dst \
-e tcp.srcport \
-e tcp.dstport \
-e tcp.len \
-e data.data
Warning: program compiled against libxml 215 using older 214
6      0.727603000      74.53.140.153      10.10.1.4      25      1470      181
7      0.732749000      10.10.1.4      74.53.140.153      1470      25      9
9      1.074123000      74.53.140.153      10.10.1.4      25      1470      137
10     1.076669000      10.10.1.4      74.53.140.153      1470      25      12
11     1.419021000      74.53.140.153      10.10.1.4      25      1470      18
12     1.419595000      10.10.1.4      74.53.140.153      1470      25      30
13     1.761484000      74.53.140.153      10.10.1.4      25      1470      18
14     1.762058000      10.10.1.4      74.53.140.153      1470      25      18
15     2.121738000      74.53.140.153      10.10.1.4      25      1470      30
16     2.122354000      10.10.1.4      74.53.140.153      1470      25      36
17     2.464705000      74.53.140.153      10.10.1.4      25      1470      8
18     2.465190000      10.10.1.4      74.53.140.153      1470      25      39
19     2.827648000      74.53.140.153      10.10.1.4      25      1470      14
20     2.828143000      10.10.1.4      74.53.140.153      1470      25      6
21     3.169619000      74.53.140.153      10.10.1.4      25      1470      56
22     3.200683000      10.10.1.4      74.53.140.153      1470      25      1460
23     3.200726000      10.10.1.4      74.53.140.153      1470      25      1460
24     3.200744000      10.10.1.4      74.53.140.153      1470      25      1460
25     3.200763000      10.10.1.4      74.53.140.153      1470      25      1460
27     3.203110000      10.10.1.4      74.53.140.153      1470      25      1452
32     3.612912000      10.10.1.4      74.53.140.153      1470      25      1452
33     3.612938000      10.10.1.4      74.53.140.153      1470      25      1452
35     3.977699000      10.10.1.4      74.53.140.153      1470      25      1452
36     3.977754000      10.10.1.4      74.53.140.153      1470      25      1452
38     4.002121000      10.10.1.4      74.53.140.153      1470      25      1452
39     4.002139000      10.10.1.4      74.53.140.153      1470      25      1452
41     4.342568000      10.10.1.4      74.53.140.153      1470      25      1452
42     4.342595000      10.10.1.4      74.53.140.153      1470      25      1452
44     4.366256000      10.10.1.4      74.53.140.153      1470      25      1452
45     4.366274000      10.10.1.4      74.53.140.153      1470      25      29
52     4.756729000      74.53.140.153      10.10.1.4      25      1470      28
54     7.271765000      10.10.1.4      74.53.140.153      1470      25      6
56     7.613407000      74.53.140.153      10.10.1.4      25      1470      48
```

Complete SMTP TCP Stream Reconstruction

The SMTP TCP stream was reconstructed to examine the complete application-layer exchange between the email client and server. The reconstruction shows the SMTP greeting, client identification, authentication exchange, sender and recipient commands, message submission, email headers, MIME structure, message body, attachment metadata, and session termination. This provides a complete view of the email transaction as transmitted over the captured SMTP connection.

The reconstructed message contains identifiable email metadata, including the sender, recipient, subject, message date, Message-ID, MIME version, content type, and Microsoft Outlook client indicator. The message also contains both plain-text and HTML representations of the email body, followed by a text-file attachment.

The important headers for our report are:

Header	Value
From	"Gurpartap Singh" <gurpartap@patriots.in>
To	<raj_deol2002in@yahoo.co.in>
Subject	SMTP
Date	Mon, 5 Oct 2009 11:36:07 +0530
Message-ID	<000301ca4581\$ef9e57f0\$cedb07d0\$@in>
MIME-Version	1.0
Content-Type	multipart/mixed
X-Mailer	Microsoft Office Outlook 12.0
Content-Language	en-us

There is also a multipart/alternative section containing:

- text/plain
- text/html

And the message contains an attachment:

filename="NEWS.txt"

Content-Type: text/plain

Content-Transfer-Encoding: quoted-printable

Content-Disposition: attachment

```

Wireshark - Follow TCP Stream (tcp.stream eq 0) - smtp_working.pcap

From: "Gurpartap Singh" <gurpartap@patriots.in>
To: <raj_deol2002in@yahoo.co.in>
Subject: SMTP
Date: Mon, 5 Oct 2009 11:36:07 +0530
Message-ID: <000301ca4581$ef9e57f0$cedb07d0$@in>
MIME-Version: 1.0
Content-Type: multipart/mixed;
boundary="-----_NextPart_000_0004_01CA45B0.095693F0"
X-Mailer: Microsoft Office Outlook 12.0
Thread-Index: AcpFgem9BvjzEDeR1Kh8i+hUyVo0A==
Content-Language: en-us
x-cr-hashedpuzzle: SeA= AAR2 ADaH Bpi0 C4G1 DigW FNb1 FPKR Fn+W HFcp HnYJ J07s
Kum6 KytW LfCi LjUt;1;cgBhAGoAXwBkAGUAbwBsADIAAAwADIAaQBuaEAAeQBhAGgAbwBvAC4AY
wBvAC4AaQBuaA==;Soshai_v1;7;{CAA37F59-1850-45C7-8540-AA27696B5398};ZwB1AHIAcABh
AHIAcABhAHIAcABhAHIAcABhAHIAcABhAHIAcABhAHIAcABhAHIAcABhAHIAcABhAHIAcABhAHIAcABh
AFQAUA==
x-cr-puzzleid: {CAA37F59-1850-45C7-8540-AA27696B5398}

This is a multipart message in MIME format.

-----_NextPart_000_0004_01CA45B0.095693F0
Content-Type: multipart/alternative;
boundary="-----_NextPart_001_0005_01CA45B0.095693F0"

-----_NextPart_001_0005_01CA45B0.095693F0
Content-Type: text/plain;
charset="us-ascii"
Content-Transfer-Encoding: 7bit

Hello

```

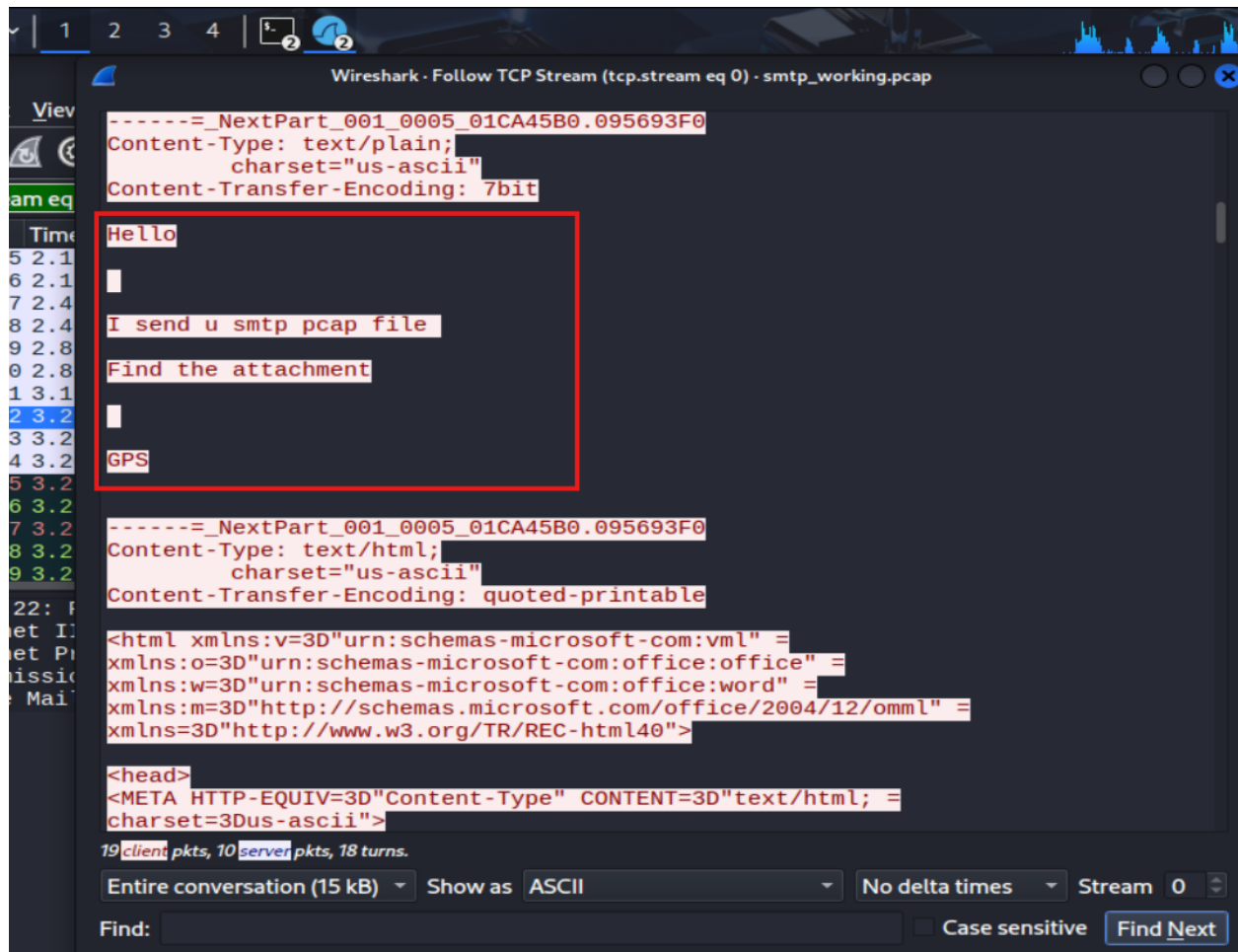
The body of the plain-text portion says:

Hello

I send u smtp pcap file

Find the attachment

GPS



One important forensic observation is that the SMTP server advertised:

250-AUTH PLAIN LOGIN

250-STARTTLS

but the captured session proceeded with: AUTH LOGIN

```
Wireshark · Follow TCP Stream (tcp.stream eq 0) - smtp_working.pcap

220-xc90.websitewelcome.com ESMTP Exim 4.69 #1 Mon, 05 Oct 2009 01:05:54 -0500
220-We do not authorize the use of this system to transport unsolicited,
220 and/or bulk e-mail.

EHLO GP

250-xc90.websitewelcome.com Hello GP [122.162.143.157]
250-SIZE 52428800
250-PIPELINING
250-AUTH PLAIN LOGIN
250-STARTTLS
250 HELP

AUTH LOGIN

334 VXNlcm5hbWU6

Z3VycGFydGFwQHhhdHJpb3RzLmlu

334 UGFzc3dvcmQ6

cHVuamFiQDEyMw==

235 Authentication succeeded

MAIL FROM: <gurpartap@patriots.in>

250 OK

RCPT TO: <raj_deol2002in@yahoo.co.in>

250 Accepted

19 client pkts, 10 server pkts, 18 turns.
Entire conversation (15 kB) Show as ASCII No delta times Stream 0
Find: Case sensitive Find Next
```

SMTP Client, Server and Network Metadata

Network and transport metadata were extracted from TCP stream 0 to establish the endpoints involved in the SMTP communication. The packet-level output identifies the Ethernet source and destination addresses, IPv4 addresses, TCP source and destination ports, frame timing, and TCP stream identifier for the captured exchange. The SMTP client is represented by IP address 10.10.1.4 and MAC address 00:e0:1c:3c:17, communicating with the SMTP server at 74.53.140.153 and MAC address 00:1f:33:d9:81:60. The client used ephemeral TCP port 1470, while the server used the standard SMTP port 25. All packets associated with the SMTP session are identified as TCP stream 0.

The exchange begins at approximately 0.037 seconds into the captured stream and continues through the SMTP session and connection termination.

```
kali-linux-2025.2-vmware... X
File Actions Edit View Help Go Capture Analyze Statistics Telephony Wireless Tools Help
kali@kali: ~/SBT-DF203-Lab4
(kali@kali)~/SBT-DF203-Lab4
$ tshark -r working/smtp_working.pcap \
-Y 'tcp.stream == 0' \
-T fields \
-E header=y \
-E separator=' \ ' \
-e frame.number \
-e frame.time_relative \
-e eth.src \
-e eth.dst \
-e ip.src \
-e ip.dst \
-e tcp.srcport \
-e tcp.dstport \
-e tcp.stream
Warning: program compiled against libxml 215 using older 214
frame.number frame.time_relative eth.src eth.dst ip.src ip.dst tcp.srcport tcp.dstport tcp.stream
3 0.036986000 00:01:33:d9:81:60 00:1f:33:d9:81:60 10.10.1.4 74.53.140.153 1470 25 0
4 0.383968000 00:01:33:d9:81:60 00:0e:1c:3c:17:c2 74.53.140.153 10.10.1.4 25 1470 0
5 0.383968000 00:01:33:d9:81:60 00:0e:1c:3c:17:c2 74.53.140.153 10.10.1.4 25 1470 0
6 0.727680000 00:01:33:d9:81:60 00:0e:1c:3c:17:c2 74.53.140.153 10.10.1.4 25 1470 0
7 0.732749000 00:01:33:d9:81:60 00:0e:1c:3c:17:c2 74.53.140.153 10.10.1.4 25 1470 0
8 1.073326000 00:01:33:d9:81:60 00:0e:1c:3c:17:c2 74.53.140.153 10.10.1.4 25 1470 0
9 1.074123000 00:01:33:d9:81:60 00:0e:1c:3c:17:c2 74.53.140.153 10.10.1.4 25 1470 0
10 1.076669000 00:01:33:d9:81:60 00:0e:1c:3c:17:c2 74.53.140.153 10.10.1.4 25 1470 0
11 1.419021000 00:01:33:d9:81:60 00:0e:1c:3c:17:c2 74.53.140.153 10.10.1.4 25 1470 0
12 1.419595000 00:01:33:d9:81:60 00:0e:1c:3c:17:c2 74.53.140.153 10.10.1.4 25 1470 0
13 1.761484000 00:01:33:d9:81:60 00:0e:1c:3c:17:c2 74.53.140.153 10.10.1.4 25 1470 0
14 1.762058000 00:01:33:d9:81:60 00:0e:1c:3c:17:c2 74.53.140.153 10.10.1.4 25 1470 0
15 2.121738000 00:01:33:d9:81:60 00:0e:1c:3c:17:c2 74.53.140.153 10.10.1.4 25 1470 0
16 2.122354000 00:01:33:d9:81:60 00:0e:1c:3c:17:c2 74.53.140.153 10.10.1.4 25 1470 0
17 2.464705000 00:01:33:d9:81:60 00:0e:1c:3c:17:c2 74.53.140.153 10.10.1.4 25 1470 0
18 2.465190000 00:01:33:d9:81:60 00:0e:1c:3c:17:c2 74.53.140.153 10.10.1.4 25 1470 0
19 2.827648000 00:01:33:d9:81:60 00:0e:1c:3c:17:c2 74.53.140.153 10.10.1.4 25 1470 0
20 2.828143000 00:01:33:d9:81:60 00:0e:1c:3c:17:c2 74.53.140.153 10.10.1.4 25 1470 0
21 3.169619000 00:01:33:d9:81:60 00:0e:1c:3c:17:c2 74.53.140.153 10.10.1.4 25 1470 0
22 3.200633000 00:01:33:d9:81:60 00:0e:1c:3c:17:c2 74.53.140.153 10.10.1.4 25 1470 0
23 3.200726000 00:01:33:d9:81:60 00:0e:1c:3c:17:c2 74.53.140.153 10.10.1.4 25 1470 0
24 3.200744000 00:01:33:d9:81:60 00:0e:1c:3c:17:c2 74.53.140.153 10.10.1.4 25 1470 0
25 3.200763000 00:01:33:d9:81:60 00:0e:1c:3c:17:c2 74.53.140.153 10.10.1.4 25 1470 0
26 3.203055000 00:01:33:d9:81:60 00:0e:1c:3c:17:c2 192.168.1.1,10.10.1.4 10.10.1.4,74.53.140.153 1470 25 0
27 3.203110000 00:01:33:d9:81:60 00:0e:1c:3c:17:c2 10.10.1.4 74.53.140.153 1470 25 0
28 3.203563000 00:01:33:d9:81:60 00:0e:1c:3c:17:c2 192.168.1.1,10.10.1.4 10.10.1.4,74.53.140.153 1470 25 0
29 3.204188000 00:01:33:d9:81:60 00:0e:1c:3c:17:c2 192.168.1.1,10.10.1.4 10.10.1.4,74.53.140.153 1470 25 0
30 3.204574000 00:01:33:d9:81:60 00:0e:1c:3c:17:c2 74.53.140.153 10.10.1.4 25 1470 0
31 3.612851000 00:01:33:d9:81:60 00:0e:1c:3c:17:c2 10.10.1.4 74.53.140.153 1470 25 0
32 3.612912000 00:01:33:d9:81:60 00:0e:1c:3c:17:c2 74.53.140.153 10.10.1.4 25 1470 0
33 3.613138000 00:01:33:d9:81:60 00:0e:1c:3c:17:c2 74.53.140.153 10.10.1.4 25 1470 0
34 3.977630000 00:01:33:d9:81:60 00:0e:1c:3c:17:c2 74.53.140.153 10.10.1.4 25 1470 0
35 3.977699000 00:01:33:d9:81:60 00:0e:1c:3c:17:c2 10.10.1.4 74.53.140.153 1470 25 0
36 3.977754000 00:01:33:d9:81:60 00:0e:1c:3c:17:c2 10.10.1.4 74.53.140.153 1470 25 0
37 4.002107000 00:01:33:d9:81:60 00:0e:1c:3c:17:c2 74.53.140.153 10.10.1.4 25 1470 0
38 4.002121000 00:01:33:d9:81:60 00:0e:1c:3c:17:c2 10.10.1.4 74.53.140.153 1470 25 0
39 4.002139000 00:01:33:d9:81:60 00:0e:1c:3c:17:c2 10.10.1.4 74.53.140.153 1470 25 0
40 4.342535000 00:01:33:d9:81:60 00:0e:1c:3c:17:c2 74.53.140.153 10.10.1.4 25 1470 0
41 4.342568000 00:01:33:d9:81:60 00:0e:1c:3c:17:c2 10.10.1.4 74.53.140.153 1470 25 0
42 4.342595000 00:01:33:d9:81:60 00:0e:1c:3c:17:c2 10.10.1.4 74.53.140.153 1470 25 0
43 4.342613000 00:01:33:d9:81:60 00:0e:1c:3c:17:c2 74.53.140.153 10.10.1.4 25 1470 0
44 4.366256000 00:01:33:d9:81:60 00:0e:1c:3c:17:c2 10.10.1.4 74.53.140.153 1470 25 0
45 4.366274000 00:01:33:d9:81:60 00:0e:1c:3c:17:c2 10.10.1.4 74.53.140.153 1470 25 0
46 4.389163000 00:01:33:d9:81:60 00:0e:1c:3c:17:c2 74.53.140.153 10.10.1.4 25 1470 0
47 4.413523000 00:01:33:d9:81:60 00:0e:1c:3c:17:c2 74.53.140.153 10.10.1.4 25 1470 0
48 4.708119000 00:01:33:d9:81:60 00:0e:1c:3c:17:c2 74.53.140.153 10.10.1.4 25 1470 0
49 4.730686000 00:01:33:d9:81:60 00:0e:1c:3c:17:c2 74.53.140.153 10.10.1.4 25 1470 0
50 4.751784000 00:01:33:d9:81:60 00:0e:1c:3c:17:c2 74.53.140.153 10.10.1.4 25 1470 0
51 4.756231000 00:01:33:d9:81:60 00:0e:1c:3c:17:c2 74.53.140.153 10.10.1.4 25 1470 0
52 4.756729000 00:01:33:d9:81:60 00:0e:1c:3c:17:c2 74.53.140.153 10.10.1.4 25 1470 0
53 4.895535000 00:01:33:d9:81:60 00:0e:1c:3c:17:c2 10.10.1.4 74.53.140.153 1470 25 0
54 7.217175000 00:01:33:d9:81:60 00:0e:1c:3c:17:c2 10.10.1.4 74.53.140.153 1470 25 0
55 7.272516000 00:01:33:d9:81:60 00:0e:1c:3c:17:c2 10.10.1.4 74.53.140.153 1470 25 0
56 7.613407000 00:01:33:d9:81:60 00:0e:1c:3c:17:c2 74.53.140.153 10.10.1.4 25 1470 0
57 7.613570000 00:01:33:d9:81:60 00:0e:1c:3c:17:c2 74.53.140.153 10.10.1.4 25 1470 0
58 7.613955000 00:01:33:d9:81:60 00:0e:1c:3c:17:c2 10.10.1.4 74.53.140.153 1470 25 0
59 7.614699000 00:01:33:d9:81:60 00:0e:1c:3c:17:c2 74.53.140.153 10.10.1.4 25 1470 0
```

```
(kali@kali)~/SBT-DF203-Lab4
$ tshark -r working/smtp_working.pcap \
-Y 'tcp.stream == 0' \
-T fields \
-E header=y \
-E separator=' \ ' \
-e frame.number \
-e frame.time_relative \
-e eth.src \
-e eth.dst \
-e ip.src \
-e ip.dst \
-e tcp.srcport \
-e tcp.dstport \
-e tcp.stream
Warning: program compiled against libxml 215 using older 214
frame.number frame.time_relative eth.src eth.dst ip.src ip.dst tcp.srcport tcp.dstport tcp.stream
3 0.036986000 00:01:33:d9:81:60 00:1f:33:d9:81:60 10.10.1.4 74.53.140.153 1470 25 0
4 0.383968000 00:01:33:d9:81:60 00:0e:1c:3c:17:c2 74.53.140.153 10.10.1.4 25 1470 0
5 0.383968000 00:01:33:d9:81:60 00:0e:1c:3c:17:c2 74.53.140.153 10.10.1.4 25 1470 0
6 0.727680000 00:01:33:d9:81:60 00:0e:1c:3c:17:c2 74.53.140.153 10.10.1.4 25 1470 0
7 0.732749000 00:01:33:d9:81:60 00:0e:1c:3c:17:c2 74.53.140.153 10.10.1.4 25 1470 0
8 1.073326000 00:01:33:d9:81:60 00:0e:1c:3c:17:c2 74.53.140.153 10.10.1.4 25 1470 0
9 1.074123000 00:01:33:d9:81:60 00:0e:1c:3c:17:c2 74.53.140.153 10.10.1.4 25 1470 0
10 1.076669000 00:01:33:d9:81:60 00:0e:1c:3c:17:c2 74.53.140.153 10.10.1.4 25 1470 0
11 1.419021000 00:01:33:d9:81:60 00:0e:1c:3c:17:c2 74.53.140.153 10.10.1.4 25 1470 0
12 1.419595000 00:01:33:d9:81:60 00:0e:1c:3c:17:c2 74.53.140.153 10.10.1.4 25 1470 0
13 1.761484000 00:01:33:d9:81:60 00:0e:1c:3c:17:c2 74.53.140.153 10.10.1.4 25 1470 0
14 1.762058000 00:01:33:d9:81:60 00:0e:1c:3c:17:c2 74.53.140.153 10.10.1.4 25 1470 0
15 2.121738000 00:01:33:d9:81:60 00:0e:1c:3c:17:c2 74.53.140.153 10.10.1.4 25 1470 0
16 2.122354000 00:01:33:d9:81:60 00:0e:1c:3c:17:c2 74.53.140.153 10.10.1.4 25 1470 0
17 2.464705000 00:01:33:d9:81:60 00:0e:1c:3c:17:c2 74.53.140.153 10.10.1.4 25 1470 0
18 2.465190000 00:01:33:d9:81:60 00:0e:1c:3c:17:c2 74.53.140.153 10.10.1.4 25 1470 0
19 2.827648000 00:01:33:d9:81:60 00:0e:1c:3c:17:c2 74.53.140.153 10.10.1.4 25 1470 0
20 2.828143000 00:01:33:d9:81:60 00:0e:1c:3c:17:c2 74.53.140.153 10.10.1.4 25 1470 0
21 3.169619000 00:01:33:d9:81:60 00:0e:1c:3c:17:c2 74.53.140.153 10.10.1.4 25 1470 0
22 3.200633000 00:01:33:d9:81:60 00:0e:1c:3c:17:c2 74.53.140.153 10.10.1.4 25 1470 0
23 3.200726000 00:01:33:d9:81:60 00:0e:1c:3c:17:c2 74.53.140.153 10.10.1.4 25 1470 0
24 3.200744000 00:01:33:d9:81:60 00:0e:1c:3c:17:c2 74.53.140.153 10.10.1.4 25 1470 0
25 3.200763000 00:01:33:d9:81:60 00:0e:1c:3c:17:c2 74.53.140.153 10.10.1.4 25 1470 0
26 3.203055000 00:01:33:d9:81:60 00:0e:1c:3c:17:c2 192.168.1.1,10.10.1.4 10.10.1.4,74.53.140.153 1470 25 0
27 3.203110000 00:01:33:d9:81:60 00:0e:1c:3c:17:c2 10.10.1.4 74.53.140.153 1470 25 0
28 3.203563000 00:01:33:d9:81:60 00:0e:1c:3c:17:c2 192.168.1.1,10.10.1.4 10.10.1.4,74.53.140.153 1470 25 0
29 3.204188000 00:01:33:d9:81:60 00:0e:1c:3c:17:c2 192.168.1.1,10.10.1.4 10.10.1.4,74.53.140.153 1470 25 0
```

File	Actions	Edit	View	Help
10	1.07669000	00:e0:1c:3c:17:c2	00:1f:33:d9:81:60	10.10.1.4 74.53.140.153 1470 25 0
11	1.419021000	00:1f:33:d9:81:60	00:e0:1c:3c:17:c2	74.53.140.153 10.10.1.4 25 1470 0
12	1.419595000	00:e0:1c:3c:17:c2	00:1f:33:d9:81:60	10.10.1.4 74.53.140.153 25 0
13	1.761484000	00:1f:33:d9:81:60	00:e0:1c:3c:17:c2	74.53.140.153 10.10.1.4 25 1470 0
14	1.762058000	00:e0:1c:3c:17:c2	00:1f:33:d9:81:60	10.10.1.4 74.53.140.153 1470 25 0
15	2.121738000	00:1f:33:d9:81:60	00:e0:1c:3c:17:c2	74.53.140.153 10.10.1.4 25 1470 0
16	2.122354000	00:e0:1c:3c:17:c2	00:1f:33:d9:81:60	10.10.1.4 74.53.140.153 1470 25 0
17	2.464705000	00:1f:33:d9:81:60	00:e0:1c:3c:17:c2	74.53.140.153 10.10.1.4 25 1470 0
18	2.465190000	00:e0:1c:3c:17:c2	00:1f:33:d9:81:60	10.10.1.4 74.53.140.153 1470 25 0
19	2.827648000	00:1f:33:d9:81:60	00:e0:1c:3c:17:c2	74.53.140.153 10.10.1.4 25 1470 0
20	2.828143000	00:e0:1c:3c:17:c2	00:1f:33:d9:81:60	10.10.1.4 74.53.140.153 1470 25 0
21	3.169619000	00:1f:33:d9:81:60	00:e0:1c:3c:17:c2	74.53.140.153 10.10.1.4 25 1470 0
22	3.200683000	00:e0:1c:3c:17:c2	00:1f:33:d9:81:60	10.10.1.4 74.53.140.153 1470 25 0
23	3.200726000	00:e0:1c:3c:17:c2	00:1f:33:d9:81:60	10.10.1.4 74.53.140.153 1470 25 0
24	3.200744000	00:e0:1c:3c:17:c2	00:1f:33:d9:81:60	10.10.1.4 74.53.140.153 1470 25 0
25	3.200763000	00:e0:1c:3c:17:c2	00:1f:33:d9:81:60	10.10.1.4 74.53.140.153 1470 25 0
26	3.203055000	00:1f:33:d9:81:60	00:e0:1c:3c:17:c2	192.168.1.1,10.10.1.4 10.10.1.4,74.53.140.153 1470 25 0
27	3.203110000	00:e0:1c:3c:17:c2	00:1f:33:d9:81:60	10.10.1.4 74.53.140.153 1470 25 0
28	3.203563000	00:1f:33:d9:81:60	00:e0:1c:3c:17:c2	192.168.1.1,10.10.1.4 10.10.1.4,74.53.140.153 1470 25 0
29	3.204188000	00:1f:33:d9:81:60	00:e0:1c:3c:17:c2	192.168.1.1,10.10.1.4 10.10.1.4,74.53.140.153 1470 25 0
30	3.204574000	00:1f:33:d9:81:60	00:e0:1c:3c:17:c2	192.168.1.1,10.10.1.4 10.10.1.4,74.53.140.153 1470 25 0
31	3.612881000	00:1f:33:d9:81:60	00:e0:1c:3c:17:c2	74.53.140.153 10.10.1.4 25 1470 0
32	3.612912000	00:e0:1c:3c:17:c2	00:1f:33:d9:81:60	10.10.1.4 74.53.140.153 1470 25 0
33	3.612938000	00:e0:1c:3c:17:c2	00:1f:33:d9:81:60	10.10.1.4 74.53.140.153 1470 25 0
34	3.977663000	00:1f:33:d9:81:60	00:e0:1c:3c:17:c2	74.53.140.153 10.10.1.4 25 1470 0
35	3.977699000	00:e0:1c:3c:17:c2	00:1f:33:d9:81:60	10.10.1.4 74.53.140.153 1470 25 0
36	3.977754000	00:e0:1c:3c:17:c2	00:1f:33:d9:81:60	10.10.1.4 74.53.140.153 1470 25 0
37	4.002107000	00:1f:33:d9:81:60	00:e0:1c:3c:17:c2	74.53.140.153 10.10.1.4 25 1470 0
38	4.002121000	00:e0:1c:3c:17:c2	00:1f:33:d9:81:60	10.10.1.4 74.53.140.153 1470 25 0
39	4.002139000	00:e0:1c:3c:17:c2	00:1f:33:d9:81:60	10.10.1.4 74.53.140.153 1470 25 0
40	4.342535000	00:1f:33:d9:81:60	00:e0:1c:3c:17:c2	74.53.140.153 10.10.1.4 25 1470 0
41	4.342568000	00:e0:1c:3c:17:c2	00:1f:33:d9:81:60	10.10.1.4 74.53.140.153 1470 25 0
42	4.342595000	00:e0:1c:3c:17:c2	00:1f:33:d9:81:60	10.10.1.4 74.53.140.153 1470 25 0
43	4.366241000	00:1f:33:d9:81:60	00:e0:1c:3c:17:c2	74.53.140.153 10.10.1.4 25 1470 0
44	4.366256000	00:e0:1c:3c:17:c2	00:1f:33:d9:81:60	10.10.1.4 74.53.140.153 1470 25 0
45	4.366274000	00:e0:1c:3c:17:c2	00:1f:33:d9:81:60	10.10.1.4 74.53.140.153 1470 25 0
46	4.389163000	00:1f:33:d9:81:60	00:e0:1c:3c:17:c2	74.53.140.153 10.10.1.4 25 1470 0
47	4.413523000	00:1f:33:d9:81:60	00:e0:1c:3c:17:c2	74.53.140.153 10.10.1.4 25 1470 0
48	4.708119000	00:1f:33:d9:81:60	00:e0:1c:3c:17:c2	74.53.140.153 10.10.1.4 25 1470 0
49	4.730686000	00:1f:33:d9:81:60	00:e0:1c:3c:17:c2	74.53.140.153 10.10.1.4 25 1470 0
50	4.754784000	00:1f:33:d9:81:60	00:e0:1c:3c:17:c2	74.53.140.153 10.10.1.4 25 1470 0
51	4.756231000	00:1f:33:d9:81:60	00:e0:1c:3c:17:c2	74.53.140.153 10.10.1.4 25 1470 0
52	4.756729000	00:1f:33:d9:81:60	00:e0:1c:3c:17:c2	74.53.140.153 10.10.1.4 25 1470 0
53	4.885535000	00:e0:1c:3c:17:c2	00:1f:33:d9:81:60	10.10.1.4 74.53.140.153 1470 25 0
54	7.271765000	00:e0:1c:3c:17:c2	00:1f:33:d9:81:60	10.10.1.4 74.53.140.153 1470 25 0
55	7.272516000	00:e0:1c:3c:17:c2	00:1f:33:d9:81:60	10.10.1.4 74.53.140.153 1470 25 0
56	7.613407000	00:1f:33:d9:81:60	00:e0:1c:3c:17:c2	74.53.140.153 10.10.1.4 25 1470 0
57	7.613939000	00:1f:33:d9:81:60	00:e0:1c:3c:17:c2	74.53.140.153 10.10.1.4 25 1470 0
58	7.613955000	00:e0:1c:3c:17:c2	00:1f:33:d9:81:60	10.10.1.4 74.53.140.153 1470 25 0
59	7.614699000	00:1f:33:d9:81:60	00:e0:1c:3c:17:c2	74.53.140.153 10.10.1.4 25 1470 0

Extracted evidence table

Metadata	Observed value
TCP stream	0
SMTP client IP	10.10.1.4
SMTP server IP	74.53.140.153
Client TCP port	1470
Server TCP port	25
Client MAC	00:e0:1c:3c:17:c2
Server MAC	00:1f:33:d9:81:60
First observed stream packet	Frame 3
First stream timestamp	0.036986 s
SMTP server port	25

Encryption and STARTTLS Analysis

The SMTP server advertised STARTTLS as an available capability in its 250 response,

but the subsequent SMTP exchange did not contain a STARTTLS command. The client instead proceeded with AUTH LOGIN, meaning the authentication exchange occurred before TLS negotiation. This is consistent with the recovered Base64 credentials and the readable SMTP message content.

```
(kali@kali)-[~/SBT-DF203-Lab4]
$ tshark -r working/smtp_working.pcap \
-Y 'smtp.req.command == "STARTTLS" || smtp.response.code == 220' \
-T fields \
-e frame.number \
-e frame.time_relative \
-e ip.src \
-e ip.dst \
-e tcp.srcport \
-e tcp.dstport \
-e smtp.req.command \
-e smtp.response.code \
-e smtp.rsp.parameter
Warning: program compiled against libxml 215 using older 214
6      0.727603000    74.53.140.153    10.10.1.4      25      1470      220      xc90.websitewelcome.com ESMTP Exim 4.69 #1 Mon, 05 Oct 2009 01:05:54 -0500 ,We do not aut
horize the use of this system to transport unsolicited, and/or bulk e-mail.
(kali@kali)-[~/SBT-DF203-Lab4]
$
```

The SMTP server advertised STARTTLS among its supported capabilities; however, no STARTTLS command was observed in the captured session. The client proceeded directly to AUTH LOGIN, followed by the transmission of Base64-encoded authentication values. No TLS negotiation was observed before authentication or message submission. Therefore, the credentials and SMTP message content were recoverable from the packet capture. The use of TCP port 25 alone was not treated as proof of plaintext; the conclusion is based on the observed protocol exchange and absence of TLS negotiation.

TLS Packet Verification

To determine whether TLS encryption was actually established during the SMTP session, a TLS-specific filter was applied to the working capture. The filter returned no packets, indicating that no TLS traffic was identified in the captured session.

```
(kali@kali)-[~/SBT-DF203-Lab4]
$ tshark -r working/smtp_working.pcap \
-Y 'tls' \
-T fields \
-e frame.number \
-e frame.time_relative \
-e ip.src \
-e ip.dst \
-e tcp.srcport \
-e tcp.dstport \
-e _ws.col.Protocol \
-e _ws.col.Info
Warning: program compiled against libxml 215 using older 214
```

STARTTLS and Authentication Verification

The TCP stream was reviewed specifically for STARTTLS and authentication activity. The server advertised STARTTLS in frame 9, but the client proceeded with AUTH

LOGIN in frame 10, and authentication was confirmed in frame 15. No STARTTLS command was observed in the session.

```
(kali@kali)~/SBT-DF203-Lab4
$ tshark -r working/smtp_working.pcap \
  -Y 'tcp.stream == 0' \
  -T fields \
  -e frame.number \
  -e _ws.col.Protocol \
  -e _ws.col.info | grep -E1 'TLS|STARTTLS|AUTH'
Warning: program compiled against libxml 215 using older 214
6 SMTP S: 220-xc98.websitewelcome.com ESMTP Exim 4.69 #1 Mon, 05 Oct 2009 01:05:54 -0500 | We do not authorize the use of this system to transport unsolicited, | and/
or bulk e-mail.
9 SMTP S: 250-xc98.websitewelcome.com Hello GP [122.162.143.157] | SIZE 52428800 | PIPELINING | AUTH PLAIN LOGIN | STARTTLS | HELP
10 SMTP C: AUTH LOGIN
15 SMTP S: 235 Authentication succeeded
```

CONCLUSION

The forensic examination of the supplied SMTP packet capture successfully identified and reconstructed an email communication session between the client at 10.10.1.4:1470 and the SMTP server at 74.53.140.153:25. The capture contained 60 packets spanning approximately 9.20 seconds, with the primary SMTP communication occurring within TCP stream 0.

The investigation established the sequence of the SMTP transaction, beginning with the server's 220 greeting, followed by the client's EHLO command, authentication using AUTH LOGIN, successful authentication with response 235, and the subsequent MAIL FROM, RCPT TO, and DATA commands. The server accepted the submitted message with a 250 OK response before the client terminated the session using QUIT, followed by the server's 221 response.

The authentication analysis demonstrated that the session transmitted authentication values using Base64 encoding. The values could be decoded offline, demonstrating that Base64 does not provide confidentiality. Because the captured session did not establish TLS before authentication, the authentication material was recoverable from the network traffic. The recovered credentials were treated as sensitive evidence and are not reproduced unnecessarily in the report.

The SMTP message was also successfully reconstructed from the TCP stream. The analysis identified the sender and recipient, the subject SMTP, message date, Message-ID, MIME information and the Microsoft Office Outlook 12.0 client indicator. The message used a multipart/mixed MIME structure containing both plain-text and HTML content and included a NEWS.txt attachment containing Dev-C++ version/change-log information.

The network analysis identified 10.10.1.4 as the SMTP client and 74.53.140.153 as the SMTP server, with TCP port 1470 used by the client and TCP port 25 used by the SMTP server. The SMTP server identified itself as Exim 4.69.

An important security finding was that the SMTP server advertised STARTTLS as an available capability, but no STARTTLS command or TLS negotiation was observed in the captured session. Instead, the client proceeded directly from EHLO to AUTH LOGIN and subsequently transmitted the email content. The absence of observed TLS negotiation, together with the successful recovery of Base64-encoded authentication material and readable SMTP message content, indicates that the captured session did not provide confidentiality for the authentication and email data.

Overall, the investigation demonstrates how SMTP network traffic can provide significant forensic evidence when captured without effective transport encryption. The examination also demonstrates the importance of preserving original evidence, verifying working copies using cryptographic hashes, reconstructing application-layer communication, and carefully handling sensitive information recovered during forensic analysis.